

Assignment_03

December 4, 2023

1 Authors

- Raphael Wäspi (18-918-938)
- Nicolas Huber (16-936-205)

```
[54]: import math
      # Imports
      import numpy as np
      import matplotlib.pyplot as plt
      import networkx as nx
      import os
      import glob
      import re
      import random
      import multiprocessing
      from statistics import mean

      import scipy.special
      from tqdm import tqdm
      import shutil
      from joblib import Parallel, delayed
      from scipy.special import factorial
      import powerlaw
      from NEMtropy import DirectedGraph, UndirectedGraph
      from NEMtropy import matrix_generator as mg
      from NEMtropy.network_functions import build_adjacency_from_edgelist
```

```
[55]: # Define colors with reduced opacity from a colormap
      COLOR_BLUE = 'dodgerblue'
      COLOR_GREEN = 'palegreen'
      COLOR_RED = 'lightcoral'
      COLOR_YELLOW = 'gold'
      COLOR_GREY = 'slategrey'
      COLOR_BLACK = 'black'
      COLOR_LIGHT_GRAY = 'lightgray'
      COLOR_DARK_GRAY = 'darkgray'

      # Define the main data directory
```

```
OUTPUT_PATH = "./output"
```

```
[56]: # Set Seed  
SEED = 42
```

```
[57]: # Prepare output folder  
  
# Delete the entire 'output' directory and its contents  
if os.path.exists(OUTPUT_PATH):  
    shutil.rmtree(OUTPUT_PATH)  
  
# Create the 'output' directory  
os.makedirs(OUTPUT_PATH)  
  
# Define subdirectories  
OUTPUT_PATH_ERGMS = os.path.join(OUTPUT_PATH, "ERGMS")  
  
# Recreate the subdirectories  
os.makedirs(OUTPUT_PATH_ERGMS)
```

```
[58]: # Define the directory containing the .gml files  
directory = 'assignment_03_data'  
  
# Use glob to get a list of all .gml files in the directory  
gml_files = glob.glob(os.path.join(directory, '*.gml'))  
  
# Init an empty dictionary for all graphs - the key is the actual name, and the  
# value is the graph data  
graphs = {}  
  
# Use tqdm to show a progress bar in Jupyter Notebook  
with tqdm(total=len(gml_files), desc="Loading GML Files") as pbar:  
    # Loop through the list of .gml files and load them  
    for file_path in gml_files:  
        with open(file_path, 'r') as file:  
            content = file.read()  
  
            # Use regex to extract the desired part  
            match = re.search(r'/(~/+)\.gml$', file_path)  
            if match:  
                graph_name = match.group(1)  
  
                # Add to the graph dictionary  
                graph = nx.read_gml(file_path)  
                graphs[graph_name] = graph
```

```

        # Print the graph name when each part is finished
        print(f"Finished loading {graph_name}")

pbar.update(1) # Update the progress bar

```

Loading GML Files: 50%| | 1/2 [00:00<00:00, 4.52it/s]

Finished loading graph_eu_airlines

Loading GML Files: 100%| | 2/2 [00:00<00:00, 2.14it/s]

Finished loading graph_power

```

[59]: # Define the directory containing the .gml files
directory = 'assignment_03_data/World_Trade_Web'

# Use glob to get a list of all .gml files in the directory
gml_files = glob.glob(os.path.join(directory, '*.graphml'))

# Init an empty dictionary for all graphs - the key is the actual name, and the
# value is the graph data
graphs_wtn = {}

# Use tqdm to show a progress bar in Jupyter Notebook
with tqdm(total=len(gml_files), desc="Loading GML WTW Files") as pbar:
    # Loop through the list of .gml files and load them
    for file_path in gml_files:
        with open(file_path, 'r') as file:
            content = file.read()

            # Use regex to extract the desired part
            match = re.search(r'/(~/)+\.txt.graphml$', file_path)
            if match:
                graph_name = match.group(1)

                # Add to the graph dictionary
                graph = nx.read_graphml(file_path)
                graphs_wtn[graph_name] = graph

                # Print the graph name when each part is finished
                print(f"Finished loading {graph_name}")

pbar.update(1) # Update the progress bar

```

Loading GML WTW Files: 9%| | 1/11 [00:00<00:03, 2.52it/s]

Finished loading WDN_1997

Loading GML WTW Files: 18%| | 2/11 [00:06<00:35, 3.92s/it]

```

Finished loading WDN_1992
Finished loading WDN_2000

Loading GML WTW Files: 45%|          | 5/11 [00:08<00:07, 1.28s/it]

Finished loading WDN_1998
Finished loading WDN_1994

Loading GML WTW Files: 55%|          | 6/11 [00:08<00:04, 1.14it/s]

Finished loading WDN_1993
Finished loading WDN_2002

Loading GML WTW Files: 73%|          | 8/11 [00:10<00:02, 1.16it/s]

Finished loading WDN_1996

Loading GML WTW Files: 82%|          | 9/11 [00:10<00:01, 1.42it/s]

Finished loading WDN_1995

Loading GML WTW Files: 91%|          | 10/11 [00:11<00:00, 1.14it/s]

Finished loading WDN_1999
Finished loading WDN_2001

Loading GML WTW Files: 100%|         | 11/11 [00:12<00:00, 1.09s/it]

```

2 I ERGMs

```
[60]: graphs
```

```
[60]: {'graph_eu_airlines': <networkx.classes.graph.Graph at 0x1a541fc90>,
      'graph_power': <networkx.classes.graph.Graph at 0x1a8817f90>}
```

```
[61]: graphs_wtn
```

```
[61]: {'WDN_1997': <networkx.classes.digraph.DiGraph at 0x1a479ee90>,
      'WDN_1992': <networkx.classes.digraph.DiGraph at 0x1a379f2d0>,
      'WDN_2000': <networkx.classes.digraph.DiGraph at 0x1a8f4fad0>,
      'WDN_1998': <networkx.classes.digraph.DiGraph at 0x1a4d3cbd0>,
      'WDN_1994': <networkx.classes.digraph.DiGraph at 0x1aa0322d0>,
      'WDN_1993': <networkx.classes.digraph.DiGraph at 0x1a4ad5690>,
      'WDN_2002': <networkx.classes.digraph.DiGraph at 0x1ad52a190>,
      'WDN_1996': <networkx.classes.digraph.DiGraph at 0x1ab3aeb50>,
      'WDN_1995': <networkx.classes.digraph.DiGraph at 0x1ad4eaa10>,
      'WDN_1999': <networkx.classes.digraph.DiGraph at 0x1b0733310>,
      'WDN_2001': <networkx.classes.digraph.DiGraph at 0x1b0fb5bd0>}
```

2.1 1. Compute the strength assortativity coefficient on the data.

Hint: Pay attention: the underlying data (WTW) are weighted and directed, meaning you should account for four different kind of assortativity: in-in, in-out, out-in, out-out.

```
[62]: def calculate_assortativity(graph):
    in_in = nx.assortativity.degree_pearson_correlation_coefficient(G=graph,
↪x='in', y='in', weight='weight')
    in_out = nx.assortativity.degree_pearson_correlation_coefficient(G=graph,
↪x='in', y='out', weight='weight')
    out_in = nx.assortativity.degree_pearson_correlation_coefficient(G=graph,
↪x='out', y='in', weight='weight')
    out_out = nx.assortativity.degree_pearson_correlation_coefficient(G=graph,
↪x='out', y='out', weight='weight')

    # Create a dictionary to store assortativity values
    assortativity_dict = {
        "In-In Assortativity": in_in,
        "In-Out Assortativity": in_out,
        "Out-In Assortativity": out_in,
        "Out-Out Assortativity": out_out
    }

    # Return dictionary with assortativity values
    return assortativity_dict
```

```
[ ]: assortativities = {}
for graph_name, graph in graphs_wtn.items():
    assortativities[graph_name] = calculate_assortativity(graph=graph)

print(assortativities)
```

```
[64]: # Extracting graph names and assortativity values
graph_names = list(assortativities.keys())
keys = list(assortativities[graph_names[0]].keys()) # Extracting keys for
↪assortativity values
values = {graph_name: [assortativities[graph_name][key] for key in keys] for
↪graph_name in graph_names}

# Creating side-by-side bar plots for each network
bar_width = 0.2 # Width of each bar
index = np.arange(len(graph_names)) # Index for the networks

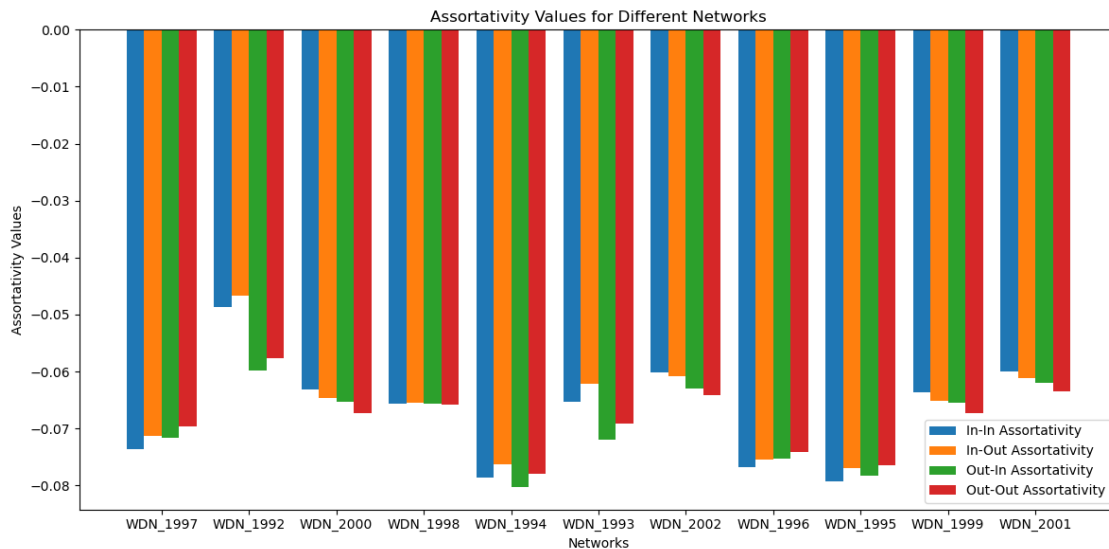
plt.figure(figsize=(12, 6))

for i, key in enumerate(keys):
    plt.bar(index + (i * bar_width), [values[graph_name][i] for graph_name in
↪graph_names], bar_width, label=key)

plt.xlabel('Networks')
plt.ylabel('Assortativity Values')
plt.title('Assortativity Values for Different Networks')
```

```
plt.xticks(index + (bar_width * (len(keys) - 1) / 2), graph_names)
plt.legend()
plt.tight_layout()

file_name=f"assortativity-values-for-internet-networks.png"
complete_plot_filename=os.path.join(OUTPUT_PATH_ERGMS, file_name)
plt.savefig(complete_plot_filename)
plt.show()
```



2.2 2. Fit the Undirected Enhanced CM and Directed Enhanced CM using the CReMa method.

Hint: You can use the nemtropy package on networkx to deploy these methods. You can find some examples at <https://github.com/nicoloval/NEMtropy/blob/master/examples/Directed%20Graphs>. ipynb. For additional questions, do not hesitate to ask at the QA.

```
[65]: # Just for understanding (from exercise)

# DIRECTED: (G: nx graph; graph_U NEMtropy graph)
# adj = nx.to_numpy_array(G)
# graph_D = DirectedGraph(adj)

# UNDIRECTED: via flipping adj matrix (G: nx graph; graph_U NEMtropy graph)
# adj = nx.to_numpy_array(G)
# adj_sym = adj + np.transpose(adj)
# graph_U = UndirectedGraph(adj_sym)
```

```
# UNDIRECTED: via to_undirected() (G: nx graph; graph_U NEMtropy graph)
# G = G.to_undirected()
# adj = nx.to_numpy_array(G)
# graph_U = UndirectedGraph(adj)
```

```
[66]: # Create undirected versions of the graph

graphs_wtn_undirected = {}
for graph_name, graph in graphs_wtn.items():
    # We can use this or either symmetrize the adj matrix
    # Since we use that, we don't need to adj_sym = adj + np.transpose(adj)
    ↪when extracting the adj from an undirected graph
    graph_undirected = graph.to_undirected()

    graphs_wtn_undirected[graph_name] = graph_undirected
```

```
[67]: FIG_SIZE = (16, 12)
      NODE_SIZE = 100
      MAX_STEPS = 10000
```

```
[ ]: graphs_wtn
```

```
[ ]: graphs_wtn_undirected
```

```
[70]: nemtropy_directed_results = {}
      nemtropy_undirected_results = {}
```

```
[71]: def to_entropy_graph(graph):
      # Extract necessary information from the NetworkX graph
      adj_weigh = nx.to_numpy_array(graph) # Adjacency matrix
      edges = list(graph.edges()) # Edge list
      degrees = dict(graph.degree()) # Degree sequence (as a dictionary)
      strengths = dict(graph.degree(weight='weight')) # Strength sequence
      ↪(assuming weights are present)

      return adj_weigh, edges, degrees, strengths
```

```
[72]: def to_networkx_graph(adj_weigh):
      graph = nx.from_numpy_array(adj_weigh, create_using=nx.DiGraph)

      # Not needed anymore
      # Set edge attributes (weights) from the adjacency matrix
      edges = graph.edges(data=True)
      for (u, v, d) in edges:
          d['weight'] = adj_weigh[u][v]
      return graph
```

```
[73]: # Helper
def plot_networkx(graph, graph_name, is_directed):

    # Plot the graph
    plt.figure(figsize=FIG_SIZE)

    # Set the layout for consistent plotting
    pos = nx.kamada_kawai_layout(graph)

    # Draw nodes and labels
    nx.draw(graph, pos, with_labels=True, node_size=NODE_SIZE,
    ↪node_color='skyblue', font_weight='bold')

    # Draw edge labels with weights
    edge_labels = nx.get_edge_attributes(graph, 'weight')
    nx.draw_networkx_edge_labels(graph, pos, edge_labels=edge_labels)

    plt.title(f'{ "Undirected" if not is_directed else "Directed" } Weighted_
    ↪Graph of Network {graph_name} (networkx plot)')
    file_name=f'{ "undirected_" if not is_directed else
    ↪"directed_" }weighted_graph_{graph_name}_networkx.png'
    complete_plot_filename=os.path.join(OUTPUT_PATH_ERGMS, file_name)
    plt.savefig(complete_plot_filename)
    plt.show()
```

```
[74]: # Helper
def plot_entropy_graph(adj_weigh, graph_name, is_directed):
    # Create a NetworkX graph from the weighted adjacency matrix
    graph = nx.from_numpy_array(adj_weigh, create_using=nx.DiGraph)

    # Set edge attributes (weights) from the adjacency matrix
    edges = graph.edges(data=True)
    for (u, v, d) in edges:
        d['weight'] = adj_weigh[u][v]

    # Plot the graph
    plt.figure(figsize=FIG_SIZE)

    pos = nx.spring_layout(graph, seed=SEED) # Set the layout for consistent_
    ↪plotting

    # Draw nodes and labels
    nx.draw(graph, pos, with_labels=True, node_size=NODE_SIZE,
    ↪node_color='skyblue', font_weight='bold')

    # Draw edge labels with weights
    edge_labels = nx.get_edge_attributes(graph, 'weight')
```



```

nx.draw_networkx_edge_labels(graph, pos, edge_labels=edge_labels)

plt.title(f'{"Undirected" if not is_directed else "Directed" } Weighted_
↳Graph of Network {graph_name} (networkx plot)')
file_name=f'{"undirected_" if not is_directed else_
↳"directed_"}weighted_graph_{graph_name}_networkx.png'
complete_plot_filename=os.path.join(OUTPUT_PATH_ERGMS, file_name)
plt.savefig(complete_plot_filename)
plt.show()

```

```

[75]: # Helper
def pretty_print_nentropy_results(results_dict):
    for graph_name, results in results_dict.items():
        nentropy_graph = results['nentropy_graph']
        fit = results['fit']
        error_degree = results['error_degree']
        error_strength = results['error_strength']
        rel_error_degree = results['relative_error_degree']
        rel_error_strength = results['relative_error_strength']

        print(f"Graph Name: {graph_name}")
        print(f"Error Degree: {error_degree}")
        print(f"Error Strength: {error_strength}")
        print(f"Relative Error Degree: {rel_error_degree}")
        print(f"Relative Error Strength: {rel_error_strength}")
        print("-----")

```

```

[ ]: for graph_name, graph in tqdm(graphs_wtn.items(), total=len(graphs_wtn.
↳items()), desc='Processing directed graphs'):

    # Plot the networkx graph
    # plot_networkx(graph=graph, graph_name=graph_name, is_directed=True)

    # Extract necessary information from the NetworkX graph
    adj_weigh, edges, degrees, strengths = to_entropy_graph(graph)

    # Instantiate DirectedGraph with the extracted information
    nentropy_graph = DirectedGraph(
        adjacency=adj_weigh,
        # Leave this out, since TA did the same
        #edgelist=edges,
        #degree_sequence=np.array(list(degrees.values())),
        #strength_sequence=np.array(list(strengths.values()))
    )

    # Plot the entropy graph

```

```

    # plot_entropy_graph(adj_weigh=adj_weigh, graph_name=graph_name,
    ↪is_directed=True)

    # We can the see the graph sequence of the graph by tapping
    #print("Out strength sequence ",nemtropy_graph.out_strength)
    #print("In strength sequence ",nemtropy_graph.in_strength)

    # Create the fit
    fit = nemtropy_graph.solve_tool(model="crema", # TODO: or dcn_exp?
        method="newton",
        initial_guess="random",
        adjacency="dcm_exp",
        method_adjacency="newton",
        max_steps=MAX_STEPS) # maybe overkill

    nemtropy_directed_results[graph_name] = {'nemtropy_graph': nemtropy_graph
    ↪, 'fit': fit, 'error_degree': nemtropy_graph.error_degree, 'error_strength':
    ↪nemtropy_graph.error_strength, 'relative_error_degree': nemtropy_graph.
    ↪relative_error_degree, 'relative_error_strength': nemtropy_graph.
    ↪relative_error_strength, }

```

```
[77]: pretty_print_nemtropy_results(nemtropy_directed_results)
```

```

Graph Name: WDN_1997
Error Degree: 3.6638425626733806e-09
Error Strength: 107564.59875488281
Relative Error Degree: 1.0703497347700856e-10
Relative Error Strength: 9.583487359438782e-06
-----
Graph Name: WDN_1992
Error Degree: 2.3851853825362923e-09
Error Strength: 343561.71673583984
Relative Error Degree: 3.910139971370971e-11
Relative Error Strength: 1.488109017333447e-06
-----
Graph Name: WDN_2000
Error Degree: 3.1629561192403344e-09
Error Strength: 303327.7660522461
Relative Error Degree: 1.317898383016806e-10
Relative Error Strength: 0.0012720940196702085
-----
Graph Name: WDN_1998
Error Degree: 2.4755024696787586e-09
Error Strength: 239821.06768798828
Relative Error Degree: 5.001115571051034e-11
Relative Error Strength: 0.0001333290305907048
-----
Graph Name: WDN_1994

```

Error Degree: 4.13637479823592e-09
Error Strength: 118861.97290039062
Relative Error Degree: 1.3787915994119733e-10
Relative Error Strength: 1.966097260638403e-06

Graph Name: WDN_1993
Error Degree: 3.208192822512501e-09
Error Strength: 122338.96789550781
Relative Error Degree: 1.188219563893519e-10
Relative Error Strength: 6.817327025395706e-05

Graph Name: WDN_2002
Error Degree: 3.6492728838766197e-09
Error Strength: 154190.61853027344
Relative Error Degree: 1.9206699388824314e-10
Relative Error Strength: 3.3974284626500237e-06

Graph Name: WDN_1996
Error Degree: 2.4397301956469164e-09
Error Strength: 118942.51055908203
Relative Error Degree: 8.132433985489721e-11
Relative Error Strength: 0.00011153452651484621

Graph Name: WDN_1995
Error Degree: 3.3731453186192084e-09
Error Strength: 341901.0478515625
Relative Error Degree: 8.059609018820755e-11
Relative Error Strength: 2.6246733303865888e-05

Graph Name: WDN_1999
Error Degree: 3.1799061162018916e-09
Error Strength: 125293.87316894531
Relative Error Degree: 3.5021615237459024e-11
Relative Error Strength: 0.0007581315300711763

Graph Name: WDN_2001
Error Degree: 5.437527761387173e-09
Error Strength: 151261.49584960938
Relative Error Degree: 2.364142504950945e-10
Relative Error Strength: 2.770415582715348e-05

```
[ ]: for graph_name, graph in tqdm(graphs_wtn_undirected.items()),  
      ↪total=len(graphs_wtn_undirected.items()), desc='Processing undirected_  
      ↪graphs'):  
  
      # Plot the networkx graph
```

```

# plot_networkx(graph=graph, graph_name=graph_name, is_directed=False)

# Extract necessary information from the NetworkX graph
adj_weigh, edges, degrees, strengths = to_entropy_graph(graph)

# Instantiate UndirectedGraph with the extracted information
nemtrophy_graph = UndirectedGraph(
    adjacency=adj_weigh,
    # Leave this out, since TA did the same
    #edgelist=edges,
    #degree_sequence=np.array(list(degrees.values())),
    #strength_sequence=np.array(list(strengths.values()))
)

# Plot the entropy graph
# plot_entropy_graph(adj_weigh=adj_weigh, graph_name=graph_name,
↪is_directed=False)
# We can see the graph sequence of the graph by tapping

# Create the fit
fit = nemtrophy_graph.solve_tool(model="cm_exp",
                                method="newton",
                                initial_guess="random",
                                adjacency="dcm_exp",
                                method_adjacency="newton",
                                max_steps=MAX_STEPS) # maybe overkill

nemtrophy_undirected_results[graph_name] = {'nemtrophy_graph': nemtrophy_graph,
↪ 'fit': fit, 'error_degree': nemtrophy_graph.error_degree, 'error_strength':
↪ nemtrophy_graph.error_strength, 'relative_error_degree': nemtrophy_graph.
↪ relative_error_degree, 'relative_error_strength': nemtrophy_graph.
↪ relative_error_strength, }

```

```
[ ]: pretty_print_nemtrophy_results(nemtrophy_undirected_results)
```

2.3 3. Sample 30 networks from the obtained ERGM distributions and measure the strength assortativity on all of them. Calculate average and standard deviation of each assortativity typology.

```
[80]: SAMPLE_SIZE = 30
```

```
[81]: def process_graph(graph_name, results, is_directed):
graph = results['nemtrophy_graph']
error_degree = results['error_degree']
error_strength = results['error_strength']

```

```

# Generate 30 ensembles copies using ensemble_sampler
output_dir = f"{OUTPUT_PATH_ERGMS}/{graph_name}-{'directed' if is_directed_
↪else 'undirected'}-sample/"
graph.ensemble_sampler(SAMPLE_SIZE, cpu_n=2, output_dir=output_dir)

sampled_graphs = []
assortativity_values = []
for i in range(0, SAMPLE_SIZE, 1):
    # We read the edgelist
    edgelist_ens = np.loadtxt(f"{output_dir}/{i}.txt")

    # and build the adjacency matrix
    ens_adj = build_adjacency_from_edgelist(edgelist=edgelist_ens,
                                             is_directed=is_directed,
                                             is_sparse=False,
                                             is_weighted=True)

    sampled_graph = to_networkx_graph(ens_adj)
    sampled_graphs.append(sampled_graph)

    # measure strength assortativity
    assortativities_sampled = calculate_assortativity(sampled_graph)
    assortativity_values.append(assortativities_sampled)

categories = list(assortativity_values[0].keys())

# Initialize dictionaries to store mean and standard deviation for each_
↪category
mean_values = {}
std_values = {}

# Loop through each category
for category in categories:
    # Extract values for the current category across all dictionaries
    category_values = [data[category] for data in assortativity_values]

    # Calculate mean and standard
    mean = np.mean(category_values)
    std = np.std(category_values)

    # Store mean and standard deviation for the category
    mean_values[category] = mean
    std_values[category] = std

results['samples'] = sampled_graphs
results['assortativity_means'] = mean_values
results['assortativity_stds'] = std_values

```

```

results['assortativity_data'] = assortativities[graph_name]

# Return the modified results dictionary
return results

```

```

[82]: def process_results_parallel(results_dict, is_directed):
        updated_results = Parallel(n_jobs=-1)(
            delayed(process_graph)(graph_name, results, is_directed)
            for graph_name, results in tqdm(results_dict.items(), desc=f'Processing_
↳{"directed" if is_directed else "undirected"} results')
        )

        # Update the original results_dict with modified values
        # Very nasty bug...
        for i, (graph_name, _) in enumerate(results_dict.items()):
            results_dict[graph_name] = updated_results[i]

        return results_dict

```

```

[83]: nemtropy_directed_results = process_results_parallel(nemtropy_directed_results,
↳is_directed=True)

```

```

Processing directed results: 100%|          | 11/11 [00:00<00:00, 23.07it/s]

```

```

[ ]: nemtropy_directed_results

```

```

[85]: nemtropy_undirected_results =
↳process_results_parallel(nemtropy_undirected_results, is_directed=False)

```

```

Processing undirected results: 100%|          | 11/11 [00:00<00:00, 456.68it/s]

```

```

[ ]: nemtropy_undirected_results

```

2.4 4. Plot strength assortativity as a function of time (the WTW network year), comparing the observed real value with the average and error bars obtained from the samples samples.

Hint: plot error bars as confidence intervals. Plot all pairs of assortativity for the undirected and directed case(in-in, in-out, out-out)

```

[ ]: assortativities

```

```

[117]: # Assortativity pairs
        assortativity_pairs = ['In-In Assortativity', 'In-Out Assortativity', 'Out-In_
↳Assortativity', 'Out-Out Assortativity']

```

```

[ ]: nemtropy_directed_results

```

```

[119]: # Years
years = list(nentropy_directed_results.keys())
x = np.arange(len(years))

# Plotting
plt.figure(figsize=(18, 6))

for i, pair in enumerate(assortativity_pairs):
    plt.subplot(1, len(assortativity_pairs), i + 1)
    sample_means = [nentropy_directed_results[year]['assortativity_means'][pair] for year in years]
    sample_std = [nentropy_directed_results[year]['assortativity_stds'][pair] for year in years]

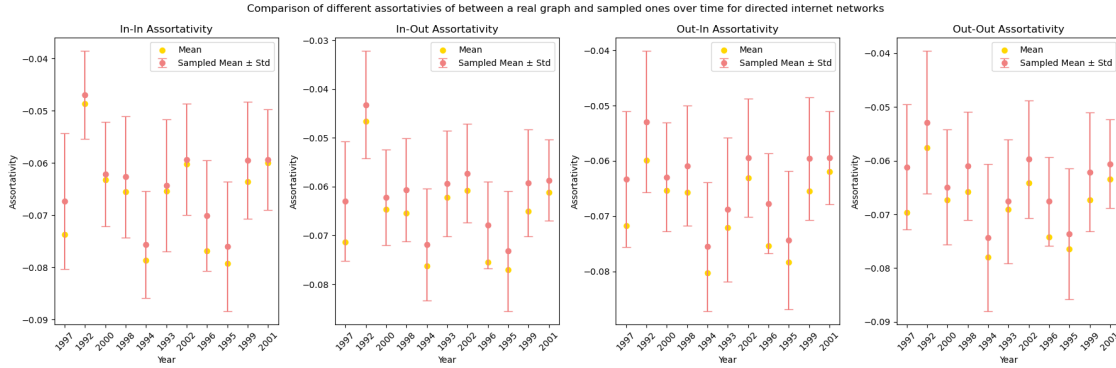
    plt.errorbar(x, sample_means, yerr=sample_std, fmt='o', label='Sampled Mean ± Std', color=COLOR_RED, capsize=5)

    real_mean = [nentropy_directed_results[year]['assortativity_data'][pair] for year in years]
    plt.scatter(x, real_mean, color=COLOR_YELLOW, label='Mean')

    plt.xticks(x, [year[-4:] for year in years], rotation=45)
    plt.title(f'{pair}')
    plt.xlabel('Year')
    plt.ylabel('Assortativity')
    plt.legend()

overall_title = "Comparison of different assortativities of between a real graph and sampled ones over time for directed internet networks"
plt.suptitle(overall_title)
plt.tight_layout()
file_name=f"assortativity-comparison-data-vs-sampled-directed-internet-networks"
complete_plot_filename=os.path.join(OUTPUT_PATH_ERGMS, file_name)
plt.savefig(complete_plot_filename)
plt.show()

```



```
[120]: # Years
years = list(nentropy_undirected_results.keys())
x = np.arange(len(years))

# Plotting
plt.figure(figsize=(18, 6))

for i, pair in enumerate(assortativity_pairs):
    plt.subplot(1, len(assortativity_pairs), i + 1)
    sample_means =
    ↪ [nentropy_undirected_results[year]['assortativity_means'][pair] for year in
    ↪ years]
    sample_std = [nentropy_undirected_results[year]['assortativity_stds'][pair]
    ↪ for year in years]

    plt.errorbar(x, sample_means, yerr=sample_std, fmt='o', label='Simulated
    ↪ Mean ± Std', color=COLOR_BLUE, capsizes=5)

    # TODO: Can we use the results from the original directed data?
    real_mean = [nentropy_directed_results[year]['assortativity_data'][pair]
    ↪ for year in years]
    plt.scatter(x, real_mean, color=COLOR_GREEN, label='Mean')

    plt.xticks(x, [year[-4:] for year in years], rotation=45)
    plt.title(f'{pair}')
    plt.xlabel('Year')
    plt.ylabel('Assortativity')
    plt.legend()

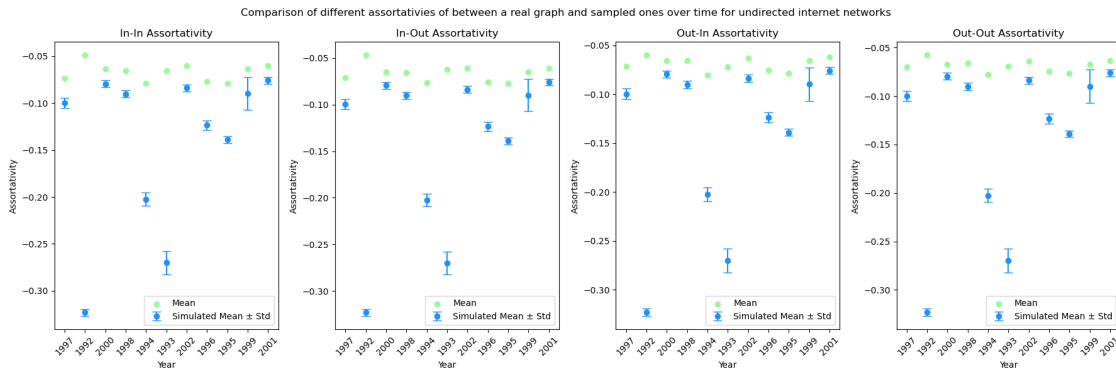
overall_title = "Comparison of different assortativities of between a real graph
    ↪ and sampled ones over time for undirected internet networks"
plt.suptitle(overall_title)
plt.tight_layout()
```



```

file_name=f"assortativity-comparison-data-vs-sampled-undirected-internet-networks"
complete_plot_filename=os.path.join(OUTPUT_PATH_ERGMS, file_name)
plt.savefig(complete_plot_filename)
plt.show()

```



We can see that all assortativities are the the same, which is expected in an undirected graph.

```

[121]: # Selecting a single assortativity pair
assortativity_pair = 'In-In Assortativity'

[122]: # Plotting
plt.figure(figsize=(8, 6))

sample_means =
    ↳ [nemtropy_undirected_results[year]['assortativity_means'][assortativity_pair]
    ↳ for year in years]
sample_std =
    ↳ [nemtropy_undirected_results[year]['assortativity_stds'][assortativity_pair]
    ↳ for year in years]

plt.errorbar(x, sample_means, yerr=sample_std, fmt='o', label='Simulated Mean ±
    ↳ Std', color=COLOR_BLUE, capsize=5)

# Using results from the original directed data
real_mean =
    ↳ [nemtropy_directed_results[year]['assortativity_data'][assortativity_pair]
    ↳ for year in years]
plt.scatter(x, real_mean, color=COLOR_GREEN, label='Mean')

plt.xticks(x, [year[-4:] for year in years], rotation=45)
plt.title(f'{assortativity_pair}')
plt.xlabel('Year')
plt.ylabel('Assortativity')
plt.legend()

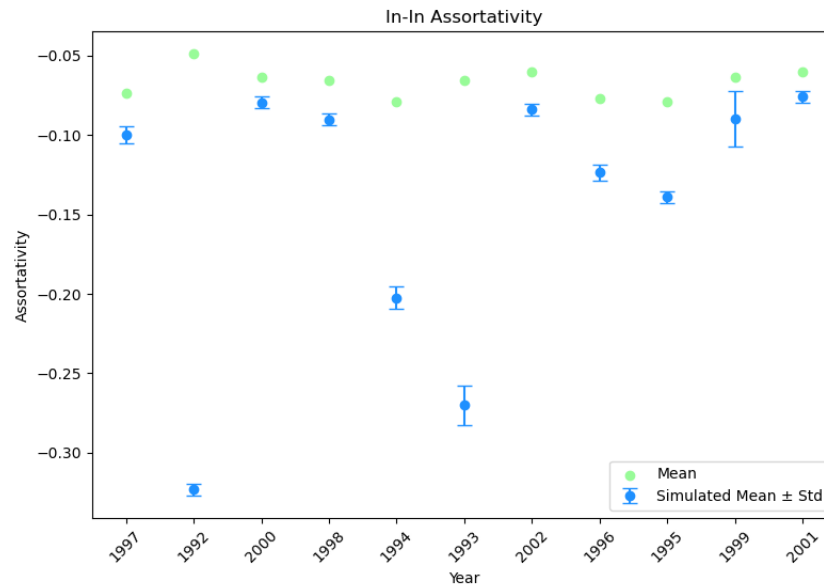
```

```

overall_title = "Comparison of different assortativities of between a real graph_
↳and sampled ones over time for undirected internet networks"
plt.suptitle(overall_title)
plt.tight_layout()
file_name = f"assortativity-comparison-{assortativity_pair.lower().replace(' ',
↳'-')}-undirected-internet-networks"
complete_plot_filename = os.path.join(OUTPUT_PATH_ERGMS, file_name)
plt.savefig(complete_plot_filename)
plt.show()

```

Comparison of different assortativities of between a real graph and sampled ones over time for undirected internet networks



```

[123]: plt.figure(figsize=(18, 6))
# Linechart
for i, pair in enumerate(assortativity_pairs):
    plt.subplot(1, len(assortativity_pairs), i + 1)
    sample_means =
↳[nentropy_directed_results[year]['assortativity_means'][pair] for year in
↳years]
    real_mean = [nentropy_directed_results[year]['assortativity_data'][pair]
↳for year in years]

    plt.plot(x, sample_means, marker='o', label='Sampled Mean',
↳color=COLOR_RED, linestyle="--")
    plt.plot(x, real_mean, marker='x', label='Real Mean', color=COLOR_YELLOW,
↳linestyle="--")

```

```

plt.xticks(x, [year[-4:] for year in years], rotation=45)
plt.title(f'{pair}')
plt.xlabel('Year')
plt.ylabel('Assortativity')
plt.legend()

overall_title = "Timeline comparison of different assortativities of between a_
↳real graph and sampled ones over time for directed internet networks"
plt.suptitle(overall_title)
plt.tight_layout()
file_name = _
↳f"assortativity-timeline-comparison-real-vs-sampled-means-directed-internet-networks"
complete_plot_filename = os.path.join(OUTPUT_PATH_ERGMS, file_name)
plt.savefig(complete_plot_filename)
plt.show()

```



```

[124]: plt.figure(figsize=(18, 6))
# Linechart
for i, pair in enumerate(assortativity_pairs):
    plt.subplot(1, len(assortativity_pairs), i + 1)
    sample_means = _
    ↳[nentropy_undirected_results[year]['assortativity_means'][pair] for year in _
    ↳years]
    real_mean = [nentropy_undirected_results[year]['assortativity_data'][pair] _
    ↳for year in years]

    plt.plot(x, sample_means, marker='o', label='Sampled Mean', _
    ↳color=COLOR_BLUE, linestyle="--")
    plt.plot(x, real_mean, marker='x', label='Real Mean', color=COLOR_GREEN, _
    ↳linestyle="--")

    plt.xticks(x, [year[-4:] for year in years], rotation=45)
    plt.title(f'{pair}')

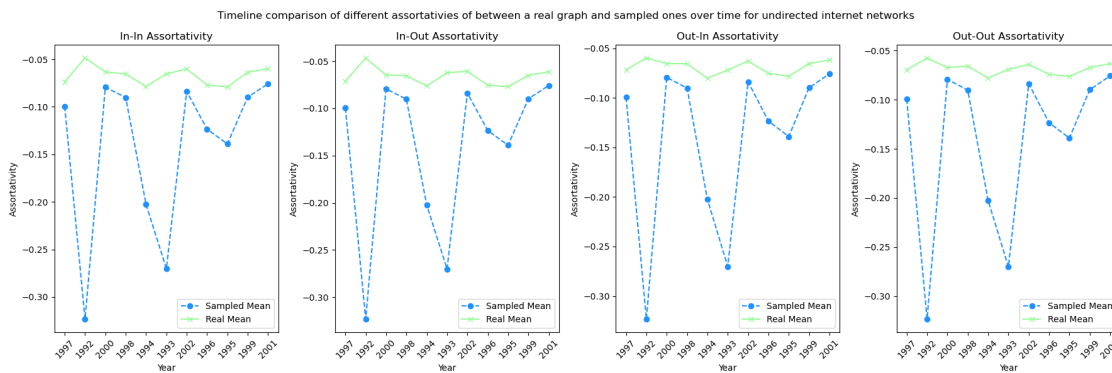
```

```

plt.xlabel('Year')
plt.ylabel('Assortativity')
plt.legend()

overall_title = "Timeline comparison of different assortativities of between a_
↳real graph and sampled ones over time for undirected internet networks"
plt.suptitle(overall_title)
plt.tight_layout()
file_name =_
↳f"assortativity-timeline-comparison-real-vs-sampled-means-undirected-internet-networks"
complete_plot_filename = os.path.join(OUTPUT_PATH_ERGMS, file_name)
plt.savefig(complete_plot_filename)
plt.show()

```



```

[129]: plt.figure(figsize=(8, 6))

sample_means =_
↳[nemtropy_undirected_results[year]['assortativity_means'][assortativity_pair]_
↳for year in years]
real_mean =_
↳[nemtropy_undirected_results[year]['assortativity_data'][assortativity_pair]_
↳for year in years]

plt.plot(x, sample_means, marker='o', label='Sampled Mean', color=COLOR_BLUE,_
↳linestyle="--")
plt.plot(x, real_mean, marker='x', label='Real Mean', color=COLOR_GREEN,_
↳linestyle="--")

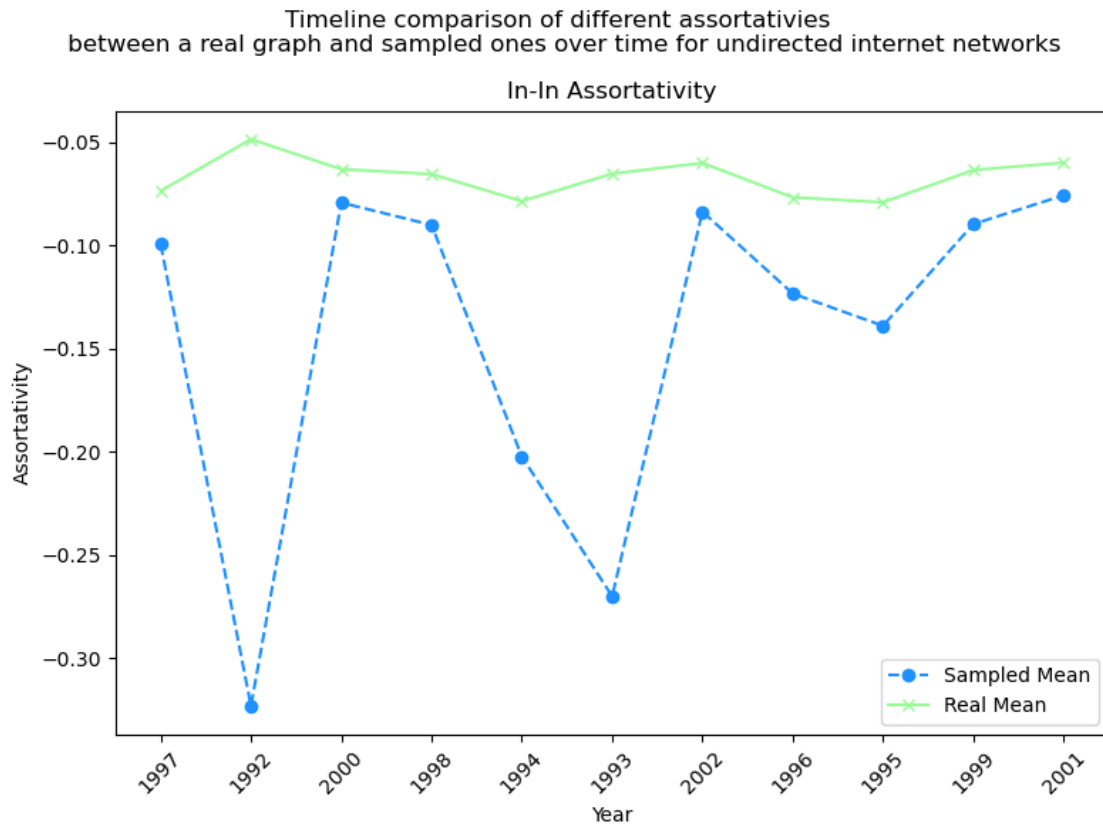
plt.xticks(x, [year[-4:] for year in years], rotation=45)
plt.title(f'{assortativity_pair}')
plt.xlabel('Year')
plt.ylabel('Assortativity')
plt.legend()

```

```

overall_title = "Timeline comparison of different assortativities \n between a_
↳real graph and sampled ones over time for undirected internet networks"
plt.suptitle(overall_title)
plt.tight_layout()
file_name = f"assortativity-timeline-comparison-{assortativity_pair.lower().
↳replace(' ', '-')}-undirected-internet-networks"
complete_plot_filename = os.path.join(OUTPUT_PATH_ERGMS, file_name)
plt.savefig(complete_plot_filename)
plt.show()

```



```

[126]: plt.figure(figsize=(12, 6))

colors = plt.cm.tab10(np.linspace(0, 1, len(assortativity_pairs)))

for i, pair in enumerate(assortativity_pairs):
    sample_means =
↳[nentropy_directed_results[year]['assortativity_means'][pair] for year in
↳years]

```

```

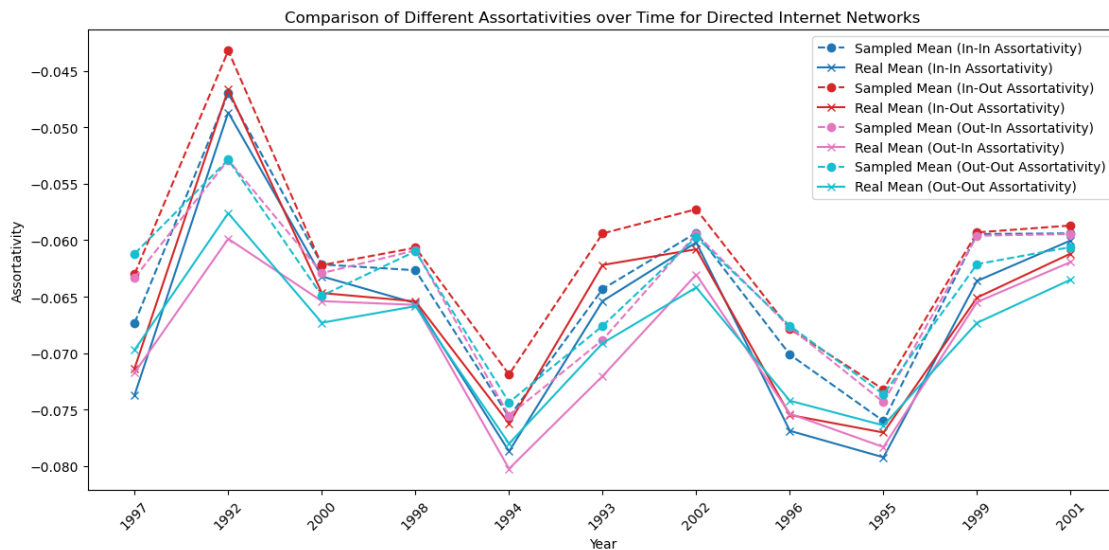
    real_mean = [nentropy_directed_results[year]['assortativity_data'][pair]
    ↪for year in years]

    plt.plot(x, sample_means, marker='o', label=f'Sampled Mean ({pair})',
    ↪color=colors[i], linestyle='--')
    plt.plot(x, real_mean, marker='x', label=f'Real Mean ({pair})',
    ↪color=colors[i], linestyle='-')

plt.xticks(x, [year[-4:] for year in years], rotation=45)
plt.xlabel('Year')
plt.ylabel('Assortativity')
plt.title('Comparison of Different Assortativities over Time for Directed
    ↪Internet Networks')
plt.legend()
plt.tight_layout()

file_name =
    ↪f"assortativity-all-comparison-real-vs-sampled-means-directed-internet-networks"
complete_plot_filename = os.path.join(OUTPUT_PATH_ERGMS, file_name)
plt.savefig(complete_plot_filename)
plt.show()

```



```

[127]: plt.figure(figsize=(12, 6))

colors = plt.cm.tab10(np.linspace(0, 1, len(assortativity_pairs)))

for i, pair in enumerate(assortativity_pairs):

```

```

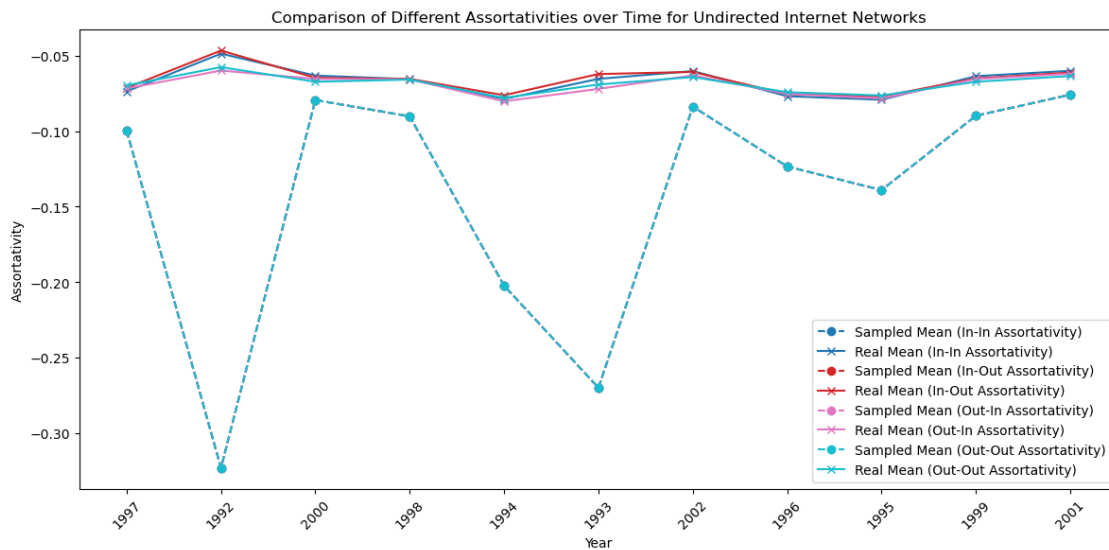
    sample_means =
    ↪[nemtropy_undirected_results[year]['assortativity_means'][pair] for year in
    ↪years]
    real_mean = [nemtropy_undirected_results[year]['assortativity_data'][pair]
    ↪for year in years]

    plt.plot(x, sample_means, marker='o', label=f'Sampled Mean ({pair})',
    ↪color=colors[i], linestyle='--')
    plt.plot(x, real_mean, marker='x', label=f'Real Mean ({pair})',
    ↪color=colors[i], linestyle='-')

plt.xticks(x, [year[-4:] for year in years], rotation=45)
plt.xlabel('Year')
plt.ylabel('Assortativity')
plt.title('Comparison of Different Assortativities over Time for Undirected
    ↪Internet Networks')
plt.legend()
plt.tight_layout()

file_name =
    ↪f"assortativity-all-comparison-real-vs-sampled-means-undirected-internet-networks"
complete_plot_filename = os.path.join(OUTPUT_PATH_ERGMS, file_name)
plt.savefig(complete_plot_filename)
plt.show()

```



```

[128]: plt.figure(figsize=(12, 6))

colors = plt.cm.tab10(np.linspace(0, 1, len(assortativity_pairs)+1))

```

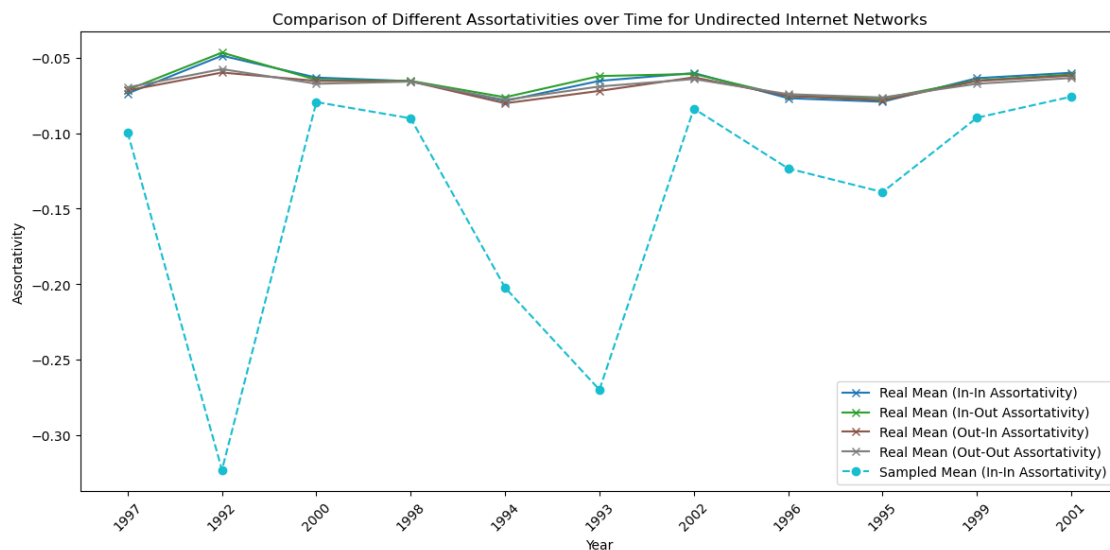
```

i = 0
for i, pair in enumerate(assortativity_pairs):
    sample_means = []
    for year in years:
        real_mean = [nemtropy_undirected_results[year]['assortativity_data'][pair]
                      for year in years]

    plt.plot(x, real_mean, marker='x', label=f'Real Mean ({pair})',
             color=colors[i], linestyle='-')
    plt.plot(x, sample_means, marker='o', label=f'Sampled Mean ({pair})',
             color=colors[i+1], linestyle='--')
plt.xticks(x, [year[-4:] for year in years], rotation=45)
plt.xlabel('Year')
plt.ylabel('Assortativity')
plt.title('Comparison of Different Assortativities over Time for Undirected Internet Networks')
plt.legend()
plt.tight_layout()

file_name = f"assortativity-reduced-comparison-real-vs-sampled-means-undirected-internet-networks"
complete_plot_filename = os.path.join(OUTPUT_PATH_ERGMS, file_name)
plt.savefig(complete_plot_filename)
plt.show()

```



2.5 5. Write a short paragraph to comment on the interpretation of strength assortativity for this dataset, and about the conclusions you can draw via the inference you made using the ERGM models.

We can observe from the plot of the previous task, that for the real data, all assortativity types are negative. This global property does describe the overall connectivity between the nodes. As we have seen in the lecture, social networks are usually assortative, whereas functional networks are usually negative assortatives i.e., disassortative. We can confirm this and this means, that means that similar vertices tend to *NOT* connect.

Taking the ERGM model into consideration, we can see, that the real mean for all assortativity types do not differ much (0.05 - 0.08). The plot *Comparison of different assortativities of between a real graph and sampled ones over time for directed internet networks* suggests, that it is possible to model the assortativity of the original directed internet networks with the NEMtropy package, since we're inside the error bars. However, the undirected case is not true, which can be seen in the plot *Comparison of different assortativities of between a real graph and sampled ones over time for undirected internet networks*. This can also be implied from the *Timeline comparison of different assortativities of between a real graph and sampled ones over time for directed internet networks*, which shows that leveraging the ERGM model, the timeline of these internet graphs is a good approximation but only for the directed version.

Creating Results: `jupyter nbconvert Assignment_03.ipynb --to pdf --output Assignment_03_Results-T1.pdf`

Assignment_03

December 4, 2023

1 Authors

- Raphael Wäspi (18-918-938)
- Nicolas Huber (16-936-205)

```
[159]: import math
# Imports
import numpy as np
import matplotlib.pyplot as plt
import networkx as nx
import os
import glob
import re
import random
from statistics import mean

import scipy.special
from tqdm import tqdm
import shutil
from joblib import Parallel, delayed
from scipy.special import factorial
import powerlaw
from NEMtropy import UndirectedGraph
```

```
[160]: # Define colors with reduced opacity from a colormap
COLOR_BLUE = 'dodgerblue'
COLOR_RED = 'lightcoral'
COLOR_GREY = 'slategrey'
COLOR_BLACK = 'black'
COLOR_LIGHT_GRAY = 'lightgray'
COLOR_DARK_GRAY = 'darkgray'

# Define the main data directory
OUTPUT_PATH = "./output"
```

```
[161]: # Prepare output folder

# Delete the entire 'output' directory and its contents
```

```

if os.path.exists(OUTPUT_PATH):
    shutil.rmtree(OUTPUT_PATH)

# Create the 'output' directory
os.makedirs(OUTPUT_PATH)

# Define subdirectories
OUTPUT_PATH_ER = os.path.join(OUTPUT_PATH, "erdos_renyi_avg_clustering")

# Recreate the subdirectories
os.makedirs(OUTPUT_PATH_ER)

```

```

[162]: # Define the directory containing the .gml files
directory = 'assignment_03_data'

# Use glob to get a list of all .gml files in the directory
gml_files = glob.glob(os.path.join(directory, '*.gml'))

# Init an empty dictionary for all graphs - the key is the actual name, and the
↳ value is the graph data
graphs = {}

# Use tqdm to show a progress bar in Jupyter Notebook
with tqdm(total=len(gml_files), desc="Loading GML Files") as pbar:
    # Loop through the list of .gml files and load them
    for file_path in gml_files:
        with open(file_path, 'r') as file:
            content = file.read()

            # Use regex to extract the desired part
            match = re.search(r'/(~/)+\.gml$', file_path)
            if match:
                graph_name = match.group(1)

                # Add to the graph dictionary
                graph = nx.read_gml(file_path)
                graphs[graph_name] = graph

                # Print the graph name when each part is finished
                print(f"Finished loading {graph_name}")

        pbar.update(1) # Update the progress bar

```

Loading GML Files: 100% | 2/2 [00:00<00:00, 79.99it/s]

```
[163]: # Define the directory containing the .gml files
directory = 'assignment_03_data/World_Trade_Web'

# Use glob to get a list of all .gml files in the directory
gml_files = glob.glob(os.path.join(directory, '*.graphml'))

# Init an empty dictionary for all graphs - the key is the actual name, and the
# value is the graph data
graphs_wtn = {}

# Use tqdm to show a progress bar in Jupyter Notebook
with tqdm(total=len(gml_files), desc="Loading GML WTW Files") as pbar:
    # Loop through the list of .gml files and load them
    for file_path in gml_files:
        with open(file_path, 'r') as file:
            content = file.read()

            # Use regex to extract the desired part
            match = re.search(r'/(~/+)\.txt.graphml$', file_path)
            if match:
                graph_name = match.group(1)

                # Add to the graph dictionary
                graph = nx.read_graphml(file_path)
                graphs_wtn[graph_name] = graph

                # Print the graph name when each part is finished
                print(f"Finished loading {graph_name}")

        pbar.update(1) # Update the progress bar
```

```
Loading GML WTW Files:  9%|          | 1/11 [00:00<00:01,  7.14it/s]
Finished loading World_Trade_Web\WDN_1992
Loading GML WTW Files: 18%|          | 2/11 [00:00<00:02,  4.01it/s]
Finished loading World_Trade_Web\WDN_1993
Finished loading World_Trade_Web\WDN_1994
Loading GML WTW Files: 36%|          | 4/11 [00:00<00:01,  3.73it/s]
Finished loading World_Trade_Web\WDN_1995
Loading GML WTW Files: 45%|          | 5/11 [00:01<00:01,  3.36it/s]
Finished loading World_Trade_Web\WDN_1996
Loading GML WTW Files: 55%|          | 6/11 [00:01<00:01,  3.10it/s]
Finished loading World_Trade_Web\WDN_1997
Loading GML WTW Files: 64%|          | 7/11 [00:02<00:01,  2.97it/s]
```

```

Finished loading World_Trade_Web\WDN_1998
Loading GML WTW Files: 73%|          | 8/11 [00:02<00:01, 2.86it/s]
Finished loading World_Trade_Web\WDN_1999
Loading GML WTW Files: 82%|          | 9/11 [00:02<00:00, 2.77it/s]
Finished loading World_Trade_Web\WDN_2000
Loading GML WTW Files: 91%|          | 10/11 [00:03<00:00, 2.54it/s]
Finished loading World_Trade_Web\WDN_2001
Loading GML WTW Files: 100%|         | 11/11 [00:03<00:00, 2.96it/s]
Finished loading World_Trade_Web\WDN_2002

```

```

[164]: # Define the directory containing the .npz files
directory = 'assignment_03_data/NYSE'

# Use glob to get a list of all .npz files in the directory
npz_files = glob.glob(os.path.join(directory, '*.npz'))

# Init an empty dictionary for all arrays - the key is the actual name, and the
# value is the array data
arrays_nyse = {}

# Use tqdm to show a progress bar in Jupyter Notebook
with tqdm(total=len(npz_files), desc="Loading NPZ Files") as pbar:
    # Loop through the list of .npz files and load them
    for file_path in npz_files:
        array_name = os.path.splitext(os.path.basename(file_path))[0]

        # Load the numpy array
        array_data = np.load(file_path)

        # Add to the array dictionary
        arrays_nyse[array_name] = array_data

        # Print the array name when each part is finished
        print(f"Finished loading {array_name}")

        pbar.update(1) # Update the progress bar

# Get stocknames
stocks = np.loadtxt(directory+"/stocknames.txt", dtype=str)

```

```

Loading NPZ Files: 100%|          | 6/6 [00:00<00:00, 103.44it/s]

```

```

Finished loading cormat_1h
Finished loading cormat_1m
Finished loading cormat_gaussian_1h
Finished loading cormat_gaussian_1m
Finished loading cormat_onefactor_1h
Finished loading cormat_onefactor_1m

```

2 Stock correlations

2.1 1. Is there a market mode at the minute and hour timescale?

Hint: The existence of the market mode is related to the gap between the largest and second largest eigenvalues of the correlation matrix.

```

[165]: # Load Data
arrays_nyse

```

```

[165]: {'cormat_1h': array([[ 1.          ,  0.16724079,  0.17792163, ...,  0.0185428 ,
        -0.04161977,  0.157766   ],
        [ 0.16724079,  1.          ,  0.46343185, ...,  0.28985369,
        0.20418909,  0.29171091],
        [ 0.17792163,  0.46343185,  1.          , ...,  0.39057488,
        0.24218336,  0.24357246],
        ...,
        [ 0.0185428 ,  0.28985369,  0.39057488, ...,  1.          ,
        0.42720044,  0.44063307],
        [-0.04161977,  0.20418909,  0.24218336, ...,  0.42720044,
        1.          ,  0.41781517],
        [ 0.157766   ,  0.29171091,  0.24357246, ...,  0.44063307,
        0.41781517,  1.          ]]),
        'cormat_1m': array([[1.          ,  0.16174328,  0.29933708, ...,  0.21205377,
        0.26330366,
        0.28766365],
        [0.16174328,  1.          ,  0.2303112 , ...,  0.18402302,  0.23935938,
        0.35348702],
        [0.29933708,  0.2303112 ,  1.          , ...,  0.2000015 ,  0.27890206,
        0.29681243],
        ...,
        [0.21205377,  0.18402302,  0.2000015 , ...,  1.          ,  0.30616488,
        0.32896516],
        [0.26330366,  0.23935938,  0.27890206, ...,  0.30616488,  1.          ,
        0.35136225],
        [0.28766365,  0.35348702,  0.29681243, ...,  0.32896516,  0.35136225,
        1.          ]]),
        'cormat_gaussian_1h': array([[ 1.          ,  0.08634609, -0.03718647, ...,
        0.08663246,

```

```

-0.00658567, -0.0686738 ],
[ 0.08634609, 1.          , 0.01719712, ..., 0.03085411,
  0.06973086, -0.07109787],
[-0.03718647, 0.01719712, 1.          , ..., 0.10206038,
  -0.03276127, 0.02083412],
...,
[ 0.08663246, 0.03085411, 0.10206038, ..., 1.          ,
  -0.01918115, 0.01378459],
[-0.00658567, 0.06973086, -0.03276127, ..., -0.01918115,
  1.          , -0.08382412],
[-0.0686738 , -0.07109787, 0.02083412, ..., 0.01378459,
  -0.08382412, 1.          ]]),
'cormat_gaussian_1m': array([[ 1.00000000e+00,  3.37684266e-03,
  7.23562750e-03, ...,
    1.92249770e-03, -9.75349240e-03, -7.36813726e-03],
[ 3.37684266e-03,  1.00000000e+00, -1.13536042e-02, ...,
    1.23984767e-03, -7.55194316e-03,  1.26137140e-03],
[ 7.23562750e-03, -1.13536042e-02,  1.00000000e+00, ...,
   -3.74411178e-03, -1.00702836e-02, -3.60383913e-03],
...,
[ 1.92249770e-03,  1.23984767e-03, -3.74411178e-03, ...,
    1.00000000e+00,  9.47256488e-03,  6.82972396e-04],
[-9.75349240e-03, -7.55194316e-03, -1.00702836e-02, ...,
    9.47256488e-03,  1.00000000e+00,  1.60163843e-02],
[-7.36813726e-03,  1.26137140e-03, -3.60383913e-03, ...,
    6.82972396e-04,  1.60163843e-02,  1.00000000e+00]]),
'cormat_onefactor_1h': array([[1.          , 0.04869414, 0.27441816, ...,
  0.16749489, 0.03936576,
    0.16454019],
[0.04869414, 1.          , 0.22243747, ..., 0.18359562, 0.03585145,
    0.30874494],
[0.27441816, 0.22243747, 1.          , ..., 0.18981075, 0.02485402,
    0.37026938],
...,
[0.16749489, 0.18359562, 0.18981075, ..., 1.          , 0.00679392,
    0.21172785],
[0.03936576, 0.03585145, 0.02485402, ..., 0.00679392, 1.          ,
    0.08217892],
[0.16454019, 0.30874494, 0.37026938, ..., 0.21172785, 0.08217892,
    1.          ]]),
'cormat_onefactor_1m': array([[1.          , 0.0778179 , 0.1258295 , ...,
  0.10320433, 0.12333375,
    0.15272495],
[0.0778179 , 1.          , 0.13371086, ..., 0.12449381, 0.11540375,
    0.17376616],
[0.1258295 , 0.13371086, 1.          , ..., 0.17169689, 0.19273065,
    0.26304523],

```

```
...,
[0.10320433, 0.12449381, 0.17169689, ..., 1.          , 0.16421046,
 0.22870027],
[0.12333375, 0.11540375, 0.19273065, ..., 0.16421046, 1.          ,
 0.23390987],
[0.15272495, 0.17376616, 0.26304523, ..., 0.22870027, 0.23390987,
 1.          ]])}
```

```
[166]: ### NOT SURE IF A LOADED THE RIGHT DATA
cormat_1h = arrays_nyse["cormat_1h"]
cormat_1m = arrays_nyse["cormat_1m"]

# Calculate eigenvalues
eigenvalues_1h = np.linalg.eigvals(cormat_1h)
eigenvalues_1m = np.linalg.eigvals(cormat_1m)

# Sort eigenvalues in descending order
sorted_eigenvalues_1h = np.sort(eigenvalues_1h)[::-1]
sorted_eigenvalues_1m = np.sort(eigenvalues_1m)[::-1]
```

```
[167]: def getAllGapValues(sorted_eigenvalues):
        result = []
        for i in range(len(sorted_eigenvalues)-1):
            result.append(sorted_eigenvalues[i]-sorted_eigenvalues[i+1])

        return result
```

```
[168]: # Get all gaps
hourly_gaps = getAllGapValues(sorted_eigenvalues_1h)
min_gaps = getAllGapValues(sorted_eigenvalues_1m)

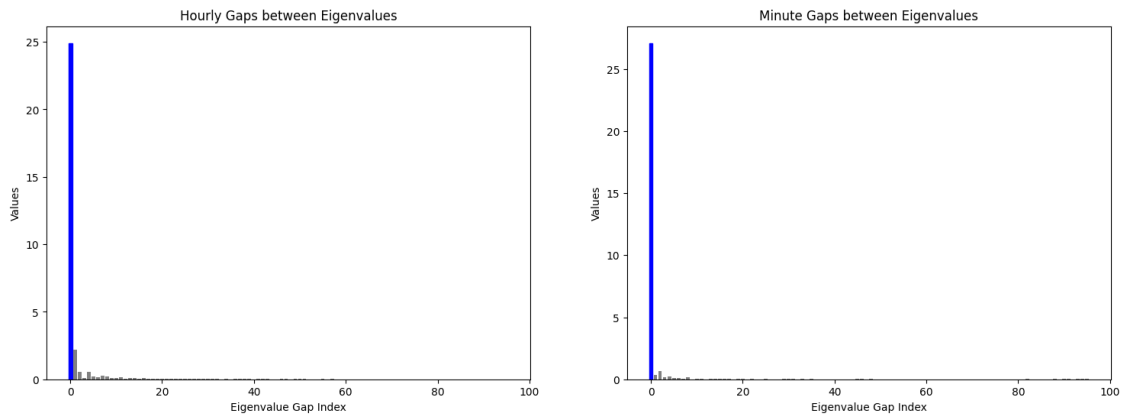
# Create a figure with two subplots side by side
fig, axs = plt.subplots(1, 2, figsize=(18, 6))

# Plot the hourly gaps in the first subplot
bars1 = axs[0].bar(range(len(hourly_gaps)), hourly_gaps, color='gray')
bars1[0].set_color('blue')
axs[0].set_xlabel('Eigenvalue Gap Index')
axs[0].set_ylabel('Values')
axs[0].set_title('Hourly Gaps between Eigenvalues')

# Plot the minute gaps in the second subplot
bars2 = axs[1].bar(range(len(min_gaps)), min_gaps, color='gray')
bars2[0].set_color('blue')
axs[1].set_xlabel('Eigenvalue Gap Index')
axs[1].set_ylabel('Values')
axs[1].set_title('Minute Gaps between Eigenvalues')
```



```
plt.show()
```



We can clearly see that the first gap is surpassing the others. This suggests the presence of a market mode at both the hour and minute timescales.

2.2 2. Compute the degree distributions of the 1 hour and 1 minute MSTs. Which are the 5 stocks with highest degree on the two different timescales MST?

Hint: To build the MST, use `networkx.minimum_spanning_tree` on the weighted undirected graph that is generated by the matrix with entries `dij` (see lecture on financial networks).

Assign the correct ticker (AAPL, AMZN, ...) as an attribute to the nodes, by using `networkx.set_node_attributes(G, values=tickers, name='ticker')`

```
[169]: cormat_1h = arrays_nyse["cormat_1h"]
       cormat_1m = arrays_nyse["cormat_1m"]
       tickers_1h = stocks
       tickers_1m = stocks
```

```
[170]: # Use MP to distinguish relevant signal from noise by identifying and removing
       ↪ noise components in the eigenvalues.
       def denoise_mp(correlation_matrix, t, noise_constant=1):

           # As said sigma can be assumed to be 1
           sigma = 1

           # Compute eigenvalues
           eigenvalues, eigenvectors = np.linalg.eig(correlation_matrix)

           # Determine the theoretical lambda_max
           #Q = len(correlation_matrix) / len(eigenvalues)
```

```

Q = t / len(correlation_matrix)
lambda_max = (sigma ** 2) * (1 + (1 / Q) + 2 * np.sqrt(1 / Q))

# Any < is classified as noise, others are signal - Replace noise s
↳with a constant
eigenvalues_denoised = np.where(eigenvalues < lambda_max, noise_constant,
↳eigenvalues)

# Renormalize eigenvalues so that sum of all eigenvalues are the length of
↳the array
eigenvalues_denoised = eigenvalues_denoised / sum(eigenvalues_denoised) *
↳len(eigenvalues_denoised)

# Undo diagonalization to get denoised correlation matrix
denoised_matrix = np.dot(eigenvectors, np.dot(np.
↳diag(eigenvalues_denoised), np.linalg.inv(eigenvectors)))

return denoised_matrix

```

```

[171]: cormat_1h_denoised = denoise_mp(cormat_1h, 22*6*60)
cormat_1m_denoised = denoise_mp(cormat_1m, 22*6*60)

num_assets_1h = len(cormat_1h_denoised)
num_assets_1m = len(cormat_1m_denoised)

# Initialize an empty distance matrix
distance_matrix_1h = np.zeros((num_assets_1h, num_assets_1h))
distance_matrix_1m = np.zeros((num_assets_1m, num_assets_1m))

# Compute the distance matrix based on the correlation coefficients
for i in range(num_assets_1h):
    for j in range(i + 1, num_assets_1h):
        correlation_coefficient = cormat_1h_denoised[i, j]
        distance_ij = np.sqrt(2 * (1 - correlation_coefficient))
        distance_matrix_1h[i, j] = distance_ij
        distance_matrix_1h[j, i] = distance_ij # Since these values should be
↳the same (symmetric). However we could only look on the left or right side
↳from the diagonal

# Compute the distance matrix based on the correlation coefficients
for i in range(num_assets_1m):
    for j in range(i + 1, num_assets_1m):
        correlation_coefficient = cormat_1m_denoised[i, j]
        distance_ij = np.sqrt(2 * (1 - correlation_coefficient))
        distance_matrix_1m[i, j] = distance_ij

```

```
distance_matrix_1m[j, i] = distance_ij # Since these values should be
↳ the same (symmetric). However we could only look on the left or right side
↳ from the diagonal
```

[172]: # Prepare the graph data

```
# Create two weighted graphs
G_1h = nx.Graph()
G_1m = nx.Graph()

# Create a dictionary with tickers as both keys and values
tickers_dict_1h = {ticker: ticker for ticker in tickers_1h}
tickers_dict_1m = {ticker: ticker for ticker in tickers_1m}

# Add nodes to the graph
G_1h.add_nodes_from(tickers_1h)
G_1m.add_nodes_from(tickers_1m)

# Set 'ticker' attribute for each node
nx.set_node_attributes(G_1h, tickers_dict_1h, name='ticker')
nx.set_node_attributes(G_1m, tickers_dict_1m, name='ticker')

# Add weighted edges for G_1h
for i in range(len(tickers_1h)):
    for j in range(i + 1, len(tickers_1h)):
        weight_1h = distance_matrix_1h[i][j]
        G_1h.add_edge(tickers_1h[i], tickers_1h[j], weight=weight_1h)

# Add weighted edges for G_1m
for i in range(len(tickers_1m)):
    for j in range(i + 1, len(tickers_1m)):
        weight_1m = distance_matrix_1m[i][j]
        G_1m.add_edge(tickers_1m[i], tickers_1m[j], weight=weight_1m)
```

[173]: # Show it graphically

```
# Make MSTs (Cheapest Path to each node)
mst_1h = nx.minimum_spanning_tree(G_1h)

# Subplot for 1h
plt.figure(figsize=(15, 6))

# Plot the G_1h
plt.subplot(1, 2, 1)
pos = nx.spring_layout(G_1h) # You can choose a different layout if needed
```

```

nx.draw(G_1h, pos, with_labels=True, node_size=800, node_color='skyblue',
        ↪font_weight='bold', edge_color='gray')
plt.title('Graph for 1 hour')

# Plot the mst_1h
plt.subplot(1, 2, 2)
pos = nx.spring_layout(mst_1h) # You can use different layout algorithms
nx.draw(mst_1h, pos, with_labels=True, font_size=8, font_color='black',
        ↪node_size=500, node_color='skyblue', edge_color='gray')
plt.title('1 hour MST')

plt.tight_layout()
plt.show()

# Make MSTs (Cheapest Path to each node)
mst_1m = nx.minimum_spanning_tree(G_1m)

# Subplot for 1h
plt.figure(figsize=(15, 6))

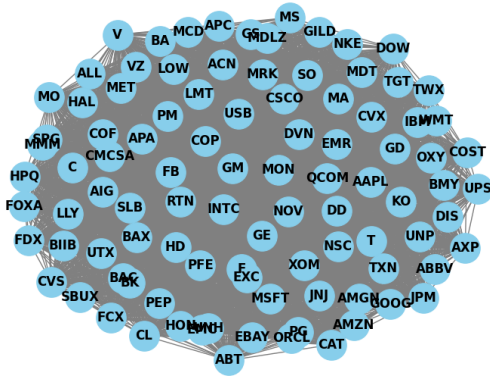
# Plot the G_1m
plt.subplot(1, 2, 1)
pos = nx.spring_layout(G_1m) # You can choose a different layout if needed
nx.draw(G_1m, pos, with_labels=True, node_size=800, node_color='skyblue',
        ↪font_weight='bold', edge_color='gray')
plt.title('Graph for 1 minute')

# Plot the mst_1h
plt.subplot(1, 2, 2)
pos = nx.spring_layout(mst_1m) # You can use different layout algorithms
nx.draw(mst_1m, pos, with_labels=True, font_size=8, font_color='black',
        ↪node_size=500, node_color='skyblue', edge_color='gray')
plt.title('1 minute MST')

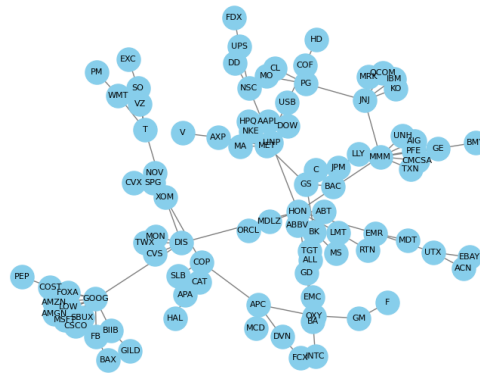
plt.tight_layout()
plt.show()

```

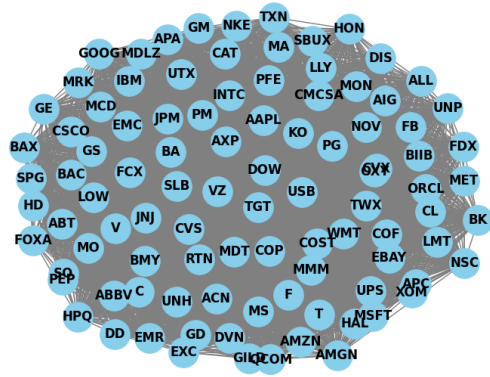
Graph for 1 hour



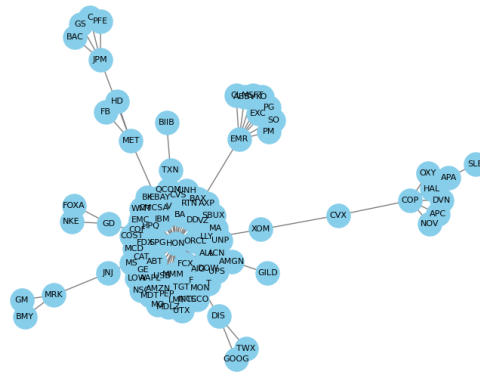
1 hour MST



Graph for 1 minute



1 minute MST



```
[174]: # Print Result

# Compute degree distributions
degree_distribution_1h = dict(mst_1h.degree())
degree_distribution_1m = dict(mst_1m.degree())

# Sort the degree distribution (dict) after descending values
sorted_dict_desc_1h = dict(sorted(degree_distribution_1h.items(), key=lambda
    ↪ item: item[1], reverse=True))
sorted_dict_desc_1m = dict(sorted(degree_distribution_1m.items(), key=lambda
    ↪ item: item[1], reverse=True))

# Print Result
print("Top 5 stocks with highest degree in 1 hour MST:",
    ↪ list(sorted_dict_desc_1h.keys())[:5])
```

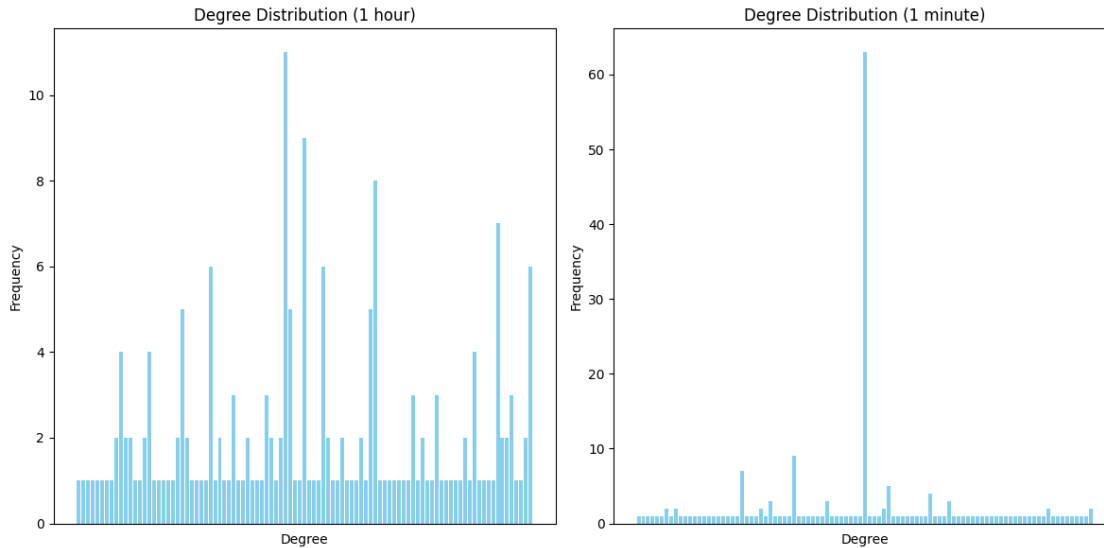
```
print("Top 5 stocks with highest degree in 1 minute MST:",  
      ↪list(sorted_dict_desc_1m.keys())[:5])
```

Top 5 stocks with highest degree in 1 hour MST: ['GOOG', 'HON', 'MMM', 'UNP', 'DIS']

Top 5 stocks with highest degree in 1 minute MST: ['HON', 'EMR', 'COP', 'JPM', 'MET']

2.3 3. Plot the degree distributions at 1 minute and 1 hour. Are the degree distributions qualitatively the same? Provide a comment to explain eventual similarities or dissimilarities.

```
[175]: # Plot degree distributions  
plt.figure(figsize=(12, 6))  
  
plt.subplot(1, 2, 1)  
plt.bar(degree_distribution_1h.keys(), degree_distribution_1h.values(),  
        ↪color='skyblue')  
plt.title('Degree Distribution (1 hour)')  
plt.xlabel('Degree')  
plt.ylabel('Frequency')  
plt.xticks([]) # Remove x-axis labels  
  
plt.subplot(1, 2, 2)  
plt.bar(degree_distribution_1m.keys(), degree_distribution_1m.values(),  
        ↪color='skyblue')  
plt.title('Degree Distribution (1 minute)')  
plt.xlabel('Degree')  
plt.ylabel('Frequency')  
plt.xticks([]) # Remove x-axis labels  
  
plt.tight_layout()  
plt.show()  
  
print("Sorted Dict which shows which node has which degree:")  
print(sorted_dict_desc_1h)  
print(sorted_dict_desc_1m)
```



Sorted Dict which shows which node has which degree:

```
{'GOOG': 11, 'HON': 9, 'MMM': 8, 'UNP': 7, 'DIS': 6, 'JNJ': 6, 'XOM': 6, 'COP': 5, 'GS': 5, 'MET': 5, 'APC': 4, 'BK': 4, 'T': 4, 'EMR': 3, 'GD': 3, 'NSC': 3, 'PG': 3, 'UTX': 3, 'APA': 2, 'AXP': 2, 'BA': 2, 'BIIB': 2, 'COF': 2, 'COST': 2, 'DVN': 2, 'FB': 2, 'GE': 2, 'GM': 2, 'JPM': 2, 'LMT': 2, 'MDLZ': 2, 'OXY': 2, 'SO': 2, 'UPS': 2, 'USB': 2, 'WMT': 2, 'AAPL': 1, 'ABV': 1, 'ABT': 1, 'ACN': 1, 'AIG': 1, 'ALL': 1, 'AMGN': 1, 'AMZN': 1, 'BAC': 1, 'BAX': 1, 'BMY': 1, 'C': 1, 'CAT': 1, 'CL': 1, 'CMCSA': 1, 'CSCO': 1, 'CVS': 1, 'CVX': 1, 'DD': 1, 'DOW': 1, 'EBAY': 1, 'EMC': 1, 'EXC': 1, 'F': 1, 'FCX': 1, 'FDX': 1, 'FOXA': 1, 'GILD': 1, 'HAL': 1, 'HD': 1, 'HPQ': 1, 'IBM': 1, 'INTC': 1, 'KO': 1, 'LLY': 1, 'LOW': 1, 'MA': 1, 'MCD': 1, 'MDT': 1, 'MO': 1, 'MON': 1, 'MRK': 1, 'MS': 1, 'MSFT': 1, 'NKE': 1, 'NOV': 1, 'ORCL': 1, 'PEP': 1, 'PFE': 1, 'PM': 1, 'QCOM': 1, 'RTN': 1, 'SBUX': 1, 'SLB': 1, 'SPG': 1, 'TGT': 1, 'TWX': 1, 'TXN': 1, 'UNH': 1, 'V': 1, 'VZ': 1}

{'HON': 63, 'EMR': 9, 'COP': 7, 'JPM': 5, 'MET': 4, 'DIS': 3, 'GD': 3, 'MRK': 3, 'AMGN': 2, 'APA': 2, 'CVX': 2, 'JNJ': 2, 'TXN': 2, 'XOM': 2, 'AAPL': 1, 'ABV': 1, 'ABT': 1, 'ACN': 1, 'AIG': 1, 'ALL': 1, 'AMZN': 1, 'APC': 1, 'AXP': 1, 'BA': 1, 'BAC': 1, 'BAX': 1, 'BIIB': 1, 'BK': 1, 'BMY': 1, 'C': 1, 'CAT': 1, 'CL': 1, 'CMCSA': 1, 'COF': 1, 'COST': 1, 'CSCO': 1, 'CVS': 1, 'DD': 1, 'DOW': 1, 'DVN': 1, 'EBAY': 1, 'EMC': 1, 'EXC': 1, 'F': 1, 'FB': 1, 'FCX': 1, 'FDX': 1, 'FOXA': 1, 'GE': 1, 'GILD': 1, 'GM': 1, 'GOOG': 1, 'GS': 1, 'HAL': 1, 'HD': 1, 'HPQ': 1, 'IBM': 1, 'INTC': 1, 'KO': 1, 'LLY': 1, 'LMT': 1, 'LOW': 1, 'MA': 1, 'MCD': 1, 'MDLZ': 1, 'MDT': 1, 'MMM': 1, 'MO': 1, 'MON': 1, 'MS': 1, 'MSFT': 1, 'NKE': 1, 'NOV': 1, 'NSC': 1, 'ORCL': 1, 'OXY': 1, 'PEP': 1, 'PFE': 1, 'PG': 1, 'PM': 1, 'QCOM': 1, 'RTN': 1, 'SBUX': 1, 'SLB': 1, 'SO': 1, 'SPG': 1, 'T': 1, 'TGT': 1, 'TWX': 1, 'UNH': 1, 'UNP': 1, 'UPS': 1, 'USB': 1, 'UTX': 1, 'V': 1, 'VZ': 1, 'WMT': 1}
```

Similarities: Both have a solid baseline of degree 1.

Dissimilarities: The minute degree plot has one clear maximum (HON) whereas the hour plot has various stocks with values over the baseline.

2.4 4. Compare the MST degree distributions of the 1 hour, 1 minute, Normal and One-Factor correlations: for each time scale, plot the three different distributions on the same plot. Is the degree distribution of MSTs well reproduced by the null Gaussian and One-Factor models?

```
[176]: len(arrays_nyse["cormat_gaussian_1h"])
```

```
[176]: 97
```

```
[177]: # Same as in 2.3 but in a function
def calculate_degree_distribution(data):
    G = nx.Graph()

    G.add_nodes_from(tickers)

    nx.set_node_attributes(G, tickers_dict, name='ticker')

    for i in range(len(tickers)):
        for j in range(i + 1, len(tickers)):
            weight = data[i][j]
            G.add_edge(tickers[i], tickers[j], weight=weight)

    mst = nx.minimum_spanning_tree(G)

    degree_distribution = dict(mst.degree())

    return degree_distribution
```

```
[178]: degree_distribution_gaussian_1h =
    ↪ calculate_degree_distribution(arrays_nyse["cormat_gaussian_1h"])
degree_distribution_gaussian_1m =
    ↪ calculate_degree_distribution(arrays_nyse["cormat_gaussian_1m"])
degree_distribution_onefactor_1h =
    ↪ calculate_degree_distribution(arrays_nyse["cormat_onefactor_1h"])
degree_distribution_onefactor_1m =
    ↪ calculate_degree_distribution(arrays_nyse["cormat_onefactor_1m"])
```

```
[181]: def plot_degree_distribution(degree_distribution, label):
    plt.hist(list(degree_distribution.values()), bins=20, alpha=0.5,
    ↪ label=label)

# Plot the degree distributions
plot_degree_distribution(degree_distribution_1m, '1 minute')
plot_degree_distribution(degree_distribution_gaussian_1m, 'Gaussian 1 minute')
```

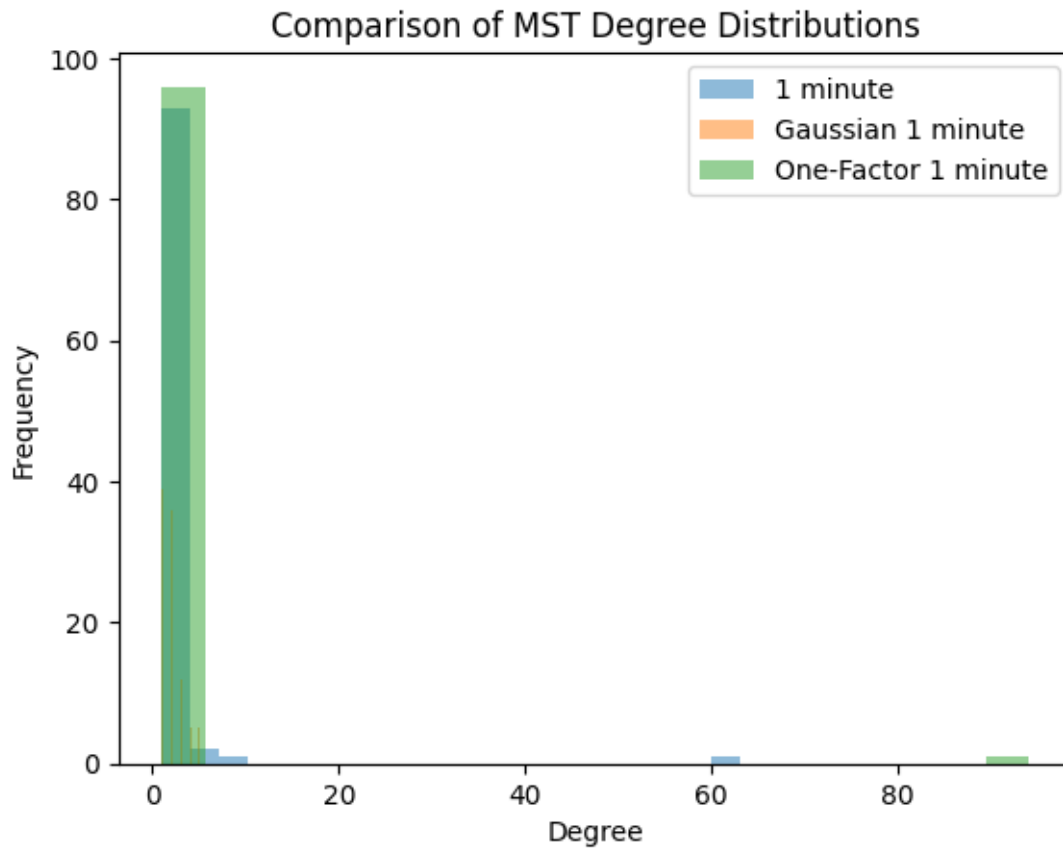


```

plot_degree_distribution(degree_distribution_onefactor_1m, 'One-Factor 1_
↪minute')

# Add labels and legend
plt.legend()
plt.xlabel('Degree')
plt.ylabel('Frequency')
plt.title('Comparison of MST Degree Distributions')
plt.show()

```



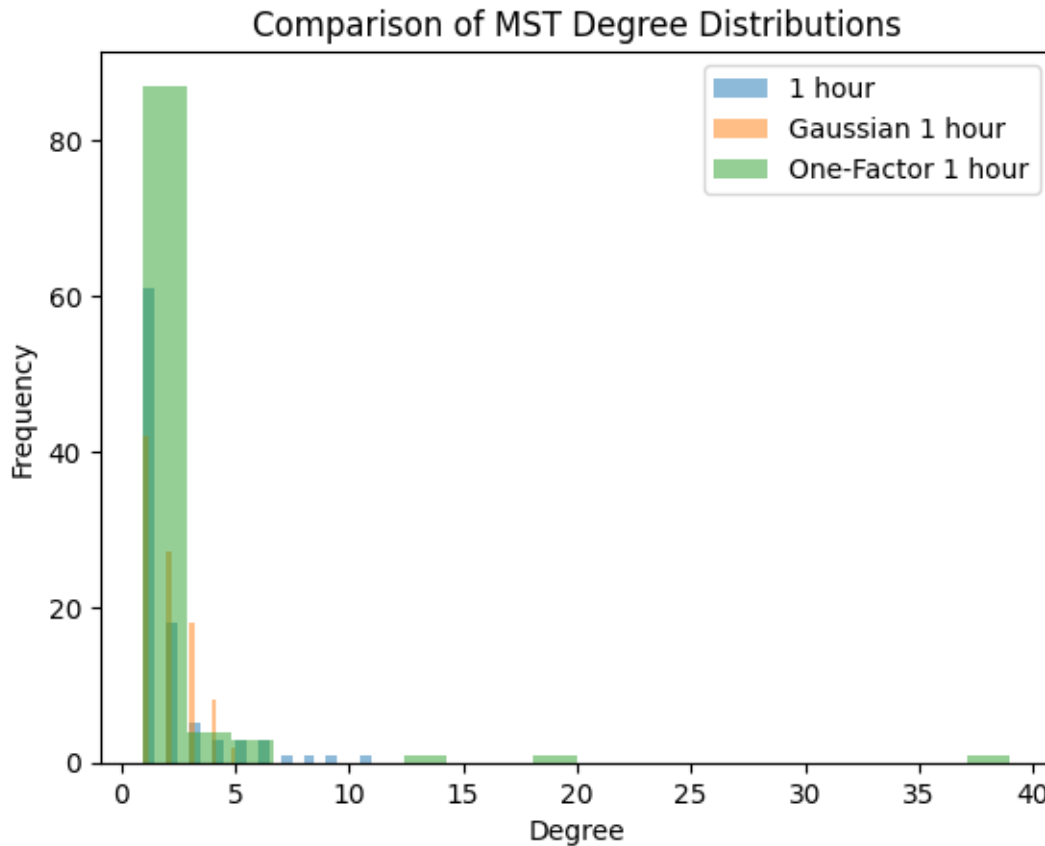
```

[182]: def plot_degree_distribution(degree_distribution, label):
        plt.hist(list(degree_distribution.values()), bins=20, alpha=0.5,
↪label=label)

# Plot the degree distributions
plot_degree_distribution(degree_distribution_1h, '1 hour')
plot_degree_distribution(degree_distribution_gaussian_1h, 'Gaussian 1 hour')
plot_degree_distribution(degree_distribution_onefactor_1h, 'One-Factor 1 hour')

```

```
# Add labels and legend
plt.legend()
plt.xlabel('Degree')
plt.ylabel('Frequency')
plt.title('Comparison of MST Degree Distributions')
plt.show()
```



```
[180]: print(dict(sorted(degree_distribution_1h.items(), key=lambda item: item[1],
    ↪reverse=True)))
print(dict(sorted(degree_distribution_1m.items(), key=lambda item: item[1],
    ↪reverse=True)))
print(dict(sorted(degree_distribution_onefactor_1h.items(), key=lambda item:
    ↪item[1], reverse=True)))
print(dict(sorted(degree_distribution_onefactor_1m.items(), key=lambda item:
    ↪item[1], reverse=True)))
print(dict(sorted(degree_distribution_gaussian_1h.items(), key=lambda item:
    ↪item[1], reverse=True)))
print(dict(sorted(degree_distribution_gaussian_1m.items(), key=lambda item:
    ↪item[1], reverse=True)))
```

```
print(dict(sorted(degree_distribution_onefactor_1h.items(), key=lambda item:
↪item[1], reverse=True)))
print(dict(sorted(degree_distribution_onefactor_1m.items(), key=lambda item:
↪item[1], reverse=True)))
```

```
{'GOOG': 11, 'HON': 9, 'MMM': 8, 'UNP': 7, 'DIS': 6, 'JNJ': 6, 'XOM': 6, 'COP':
5, 'GS': 5, 'MET': 5, 'APC': 4, 'BK': 4, 'T': 4, 'EMR': 3, 'GD': 3, 'NSC': 3,
'PG': 3, 'UTX': 3, 'APA': 2, 'AXP': 2, 'BA': 2, 'BIIB': 2, 'COF': 2, 'COST': 2,
'DVN': 2, 'FB': 2, 'GE': 2, 'GM': 2, 'JPM': 2, 'LMT': 2, 'MDLZ': 2, 'OXY': 2,
'SO': 2, 'UPS': 2, 'USB': 2, 'WMT': 2, 'AAPL': 1, 'ABV': 1, 'ABT': 1, 'ACN': 1,
'AIG': 1, 'ALL': 1, 'AMGN': 1, 'AMZN': 1, 'BAC': 1, 'BAX': 1, 'BMY': 1, 'C': 1,
'CAT': 1, 'CL': 1, 'CMCSA': 1, 'CSCO': 1, 'CVS': 1, 'CVX': 1, 'DD': 1, 'DOW': 1,
'EBAY': 1, 'EMC': 1, 'EXC': 1, 'F': 1, 'FCX': 1, 'FDX': 1, 'FOXA': 1, 'GILD': 1,
'HAL': 1, 'HD': 1, 'HPQ': 1, 'IBM': 1, 'INTC': 1, 'KO': 1, 'LLY': 1, 'LOW': 1,
'MA': 1, 'MCD': 1, 'MDT': 1, 'MO': 1, 'MON': 1, 'MRK': 1, 'MS': 1, 'MSFT': 1,
'NKE': 1, 'NOV': 1, 'ORCL': 1, 'PEP': 1, 'PFE': 1, 'PM': 1, 'QCOM': 1, 'RTN': 1,
'SBUX': 1, 'SLB': 1, 'SPG': 1, 'TGT': 1, 'TWX': 1, 'TXN': 1, 'UNH': 1, 'V': 1,
'VZ': 1}
```

```
{'HON': 63, 'EMR': 9, 'COP': 7, 'JPM': 5, 'MET': 4, 'DIS': 3, 'GD': 3, 'MRK': 3,
'AMGN': 2, 'APA': 2, 'CVX': 2, 'JNJ': 2, 'TXN': 2, 'XOM': 2, 'AAPL': 1, 'ABV':
1, 'ABT': 1, 'ACN': 1, 'AIG': 1, 'ALL': 1, 'AMZN': 1, 'APC': 1, 'AXP': 1, 'BA':
1, 'BAC': 1, 'BAX': 1, 'BIIB': 1, 'BK': 1, 'BMY': 1, 'C': 1, 'CAT': 1, 'CL': 1,
'CMCSA': 1, 'COF': 1, 'COST': 1, 'CSCO': 1, 'CVS': 1, 'DD': 1, 'DOW': 1, 'DVN':
1, 'EBAY': 1, 'EMC': 1, 'EXC': 1, 'F': 1, 'FB': 1, 'FCX': 1, 'FDX': 1, 'FOXA':
1, 'GE': 1, 'GILD': 1, 'GM': 1, 'GOOG': 1, 'GS': 1, 'HAL': 1, 'HD': 1, 'HPQ': 1,
'IBM': 1, 'INTC': 1, 'KO': 1, 'LLY': 1, 'LMT': 1, 'LOW': 1, 'MA': 1, 'MCD': 1,
'MDLZ': 1, 'MDT': 1, 'MMM': 1, 'MO': 1, 'MON': 1, 'MS': 1, 'MSFT': 1, 'NKE': 1,
'NOV': 1, 'NSC': 1, 'ORCL': 1, 'OXY': 1, 'PEP': 1, 'PFE': 1, 'PG': 1, 'PM': 1,
'QCOM': 1, 'RTN': 1, 'SBUX': 1, 'SLB': 1, 'SO': 1, 'SPG': 1, 'T': 1, 'TGT': 1,
'TWX': 1, 'UNH': 1, 'UNP': 1, 'UPS': 1, 'USB': 1, 'UTX': 1, 'V': 1, 'VZ': 1,
'WMT': 1}
```

```
{'PM': 39, 'WMT': 19, 'BAX': 13, 'EBAY': 6, 'IBM': 5, 'MA': 5, 'KO': 4, 'NKE':
4, 'F': 3, 'MCD': 3, 'AAPL': 2, 'CVX': 2, 'FDX': 2, 'T': 2, 'ABV': 1, 'ABT': 1,
'ACN': 1, 'AIG': 1, 'ALL': 1, 'AMGN': 1, 'AMZN': 1, 'APA': 1, 'APC': 1, 'AXP':
1, 'BA': 1, 'BAC': 1, 'BIIB': 1, 'BK': 1, 'BMY': 1, 'C': 1, 'CAT': 1, 'CL': 1,
'CMCSA': 1, 'COF': 1, 'COP': 1, 'COST': 1, 'CSCO': 1, 'CVS': 1, 'DD': 1, 'DIS':
1, 'DOW': 1, 'DVN': 1, 'EMC': 1, 'EMR': 1, 'EXC': 1, 'FB': 1, 'FCX': 1, 'FOXA':
1, 'GD': 1, 'GE': 1, 'GILD': 1, 'GM': 1, 'GOOG': 1, 'GS': 1, 'HAL': 1, 'HD': 1,
'HON': 1, 'HPQ': 1, 'INTC': 1, 'JNJ': 1, 'JPM': 1, 'LLY': 1, 'LMT': 1, 'LOW': 1,
'MDLZ': 1, 'MDT': 1, 'MET': 1, 'MMM': 1, 'MO': 1, 'MON': 1, 'MRK': 1, 'MS': 1,
'MSFT': 1, 'NOV': 1, 'NSC': 1, 'ORCL': 1, 'OXY': 1, 'PEP': 1, 'PFE': 1, 'PG': 1,
'QCOM': 1, 'RTN': 1, 'SBUX': 1, 'SLB': 1, 'SO': 1, 'SPG': 1, 'TGT': 1, 'TWX': 1,
'TXN': 1, 'UNH': 1, 'UNP': 1, 'UPS': 1, 'USB': 1, 'UTX': 1, 'V': 1, 'VZ': 1,
'XOM': 1}
```

```
{'F': 94, 'MA': 3, 'AAPL': 1, 'ABV': 1, 'ABT': 1, 'ACN': 1, 'AIG': 1, 'ALL': 1,
'AMGN': 1, 'AMZN': 1, 'APA': 1, 'APC': 1, 'AXP': 1, 'BA': 1, 'BAC': 1, 'BAX': 1,
'BIIB': 1, 'BK': 1, 'BMY': 1, 'C': 1, 'CAT': 1, 'CL': 1, 'CMCSA': 1, 'COF': 1,
'COP': 1, 'COST': 1, 'CSCO': 1, 'CVS': 1, 'CVX': 1, 'DD': 1, 'DIS': 1, 'DOW': 1,
```

'DVN': 1, 'EBAY': 1, 'EMC': 1, 'EMR': 1, 'EXC': 1, 'FB': 1, 'FCX': 1, 'FDX': 1,
 'FOXA': 1, 'GD': 1, 'GE': 1, 'GILD': 1, 'GM': 1, 'GOOG': 1, 'GS': 1, 'HAL': 1,
 'HD': 1, 'HON': 1, 'HPQ': 1, 'IBM': 1, 'INTC': 1, 'JNJ': 1, 'JPM': 1, 'KO': 1,
 'LLY': 1, 'LMT': 1, 'LOW': 1, 'MCD': 1, 'MDLZ': 1, 'MDT': 1, 'MET': 1, 'MMM': 1,
 'MO': 1, 'MON': 1, 'MRK': 1, 'MS': 1, 'MSFT': 1, 'NKE': 1, 'NOV': 1, 'NSC': 1,
 'ORCL': 1, 'OXY': 1, 'PEP': 1, 'PFE': 1, 'PG': 1, 'PM': 1, 'QCOM': 1, 'RTN': 1,
 'SBUX': 1, 'SLB': 1, 'SO': 1, 'SPG': 1, 'T': 1, 'TGT': 1, 'TWX': 1, 'TXN': 1,
 'UNH': 1, 'UNP': 1, 'UPS': 1, 'USB': 1, 'UTX': 1, 'V': 1, 'VZ': 1, 'WMT': 1,
 'XOM': 1}
 {'PG': 5, 'TWX': 5, 'FOXA': 4, 'GILD': 4, 'INTC': 4, 'MCD': 4, 'MON': 4, 'MS':
 4, 'MSFT': 4, 'UNH': 4, 'AIG': 3, 'AMGN': 3, 'AMZN': 3, 'BA': 3, 'BAC': 3,
 'BAX': 3, 'BK': 3, 'BMY': 3, 'CVX': 3, 'DIS': 3, 'DOW': 3, 'HAL': 3, 'JPM': 3,
 'LMT': 3, 'MA': 3, 'PEP': 3, 'QCOM': 3, 'SLB': 3, 'AAPL': 2, 'ACN': 2, 'ALL': 2,
 'BIIB': 2, 'C': 2, 'CAT': 2, 'CL': 2, 'COF': 2, 'COST': 2, 'CVS': 2, 'EBAY': 2,
 'EMC': 2, 'FDX': 2, 'JNJ': 2, 'LLY': 2, 'MDLZ': 2, 'MMM': 2, 'NKE': 2, 'NOV': 2,
 'NSC': 2, 'SO': 2, 'T': 2, 'TXN': 2, 'UNP': 2, 'USB': 2, 'UTX': 2, 'VZ': 2,
 'ABBV': 1, 'ABT': 1, 'APA': 1, 'APC': 1, 'AXP': 1, 'CMCSA': 1, 'COP': 1, 'CSCO':
 1, 'DD': 1, 'DVN': 1, 'EMR': 1, 'EXC': 1, 'F': 1, 'FB': 1, 'FCX': 1, 'GD': 1,
 'GE': 1, 'GM': 1, 'GOOG': 1, 'GS': 1, 'HD': 1, 'HON': 1, 'HPQ': 1, 'IBM': 1,
 'KO': 1, 'LOW': 1, 'MDT': 1, 'MET': 1, 'MO': 1, 'MRK': 1, 'ORCL': 1, 'OXY': 1,
 'PFE': 1, 'PM': 1, 'RTN': 1, 'SBUX': 1, 'SPG': 1, 'TGT': 1, 'UPS': 1, 'V': 1,
 'WMT': 1, 'XOM': 1}
 {'DD': 5, 'JNJ': 5, 'TGT': 5, 'TXN': 5, 'UTX': 5, 'AMZN': 4, 'EMR': 4, 'GE': 4,
 'IBM': 4, 'USB': 4, 'APA': 3, 'APC': 3, 'BK': 3, 'COF': 3, 'DVN': 3, 'F': 3,
 'GM': 3, 'INTC': 3, 'NKE': 3, 'TWX': 3, 'UNH': 3, 'V': 3, 'AAPL': 2, 'ABT': 2,
 'AIG': 2, 'ALL': 2, 'AXP': 2, 'BAC': 2, 'BAX': 2, 'C': 2, 'CL': 2, 'CMCSA': 2,
 'COP': 2, 'CVS': 2, 'EMC': 2, 'EXC': 2, 'FB': 2, 'FCX': 2, 'FOXA': 2, 'GOOG': 2,
 'GS': 2, 'HD': 2, 'HON': 2, 'JPM': 2, 'LLY': 2, 'LMT': 2, 'MCD': 2, 'MDLZ': 2,
 'MET': 2, 'MRK': 2, 'PEP': 2, 'PG': 2, 'PM': 2, 'SLB': 2, 'SO': 2, 'T': 2,
 'UPS': 2, 'VZ': 2, 'ABBV': 1, 'ACN': 1, 'AMGN': 1, 'BA': 1, 'BIIB': 1, 'BMY': 1,
 'CAT': 1, 'COST': 1, 'CSCO': 1, 'CVX': 1, 'DIS': 1, 'DOW': 1, 'EBAY': 1, 'FDX':
 1, 'GD': 1, 'GILD': 1, 'HAL': 1, 'HPQ': 1, 'KO': 1, 'LOW': 1, 'MA': 1, 'MDT': 1,
 'MMM': 1, 'MO': 1, 'MON': 1, 'MS': 1, 'MSFT': 1, 'NOV': 1, 'NSC': 1, 'ORCL': 1,
 'OXY': 1, 'PFE': 1, 'QCOM': 1, 'RTN': 1, 'SBUX': 1, 'SPG': 1, 'UNP': 1, 'WMT':
 1, 'XOM': 1}
 {'PM': 39, 'WMT': 19, 'BAX': 13, 'EBAY': 6, 'IBM': 5, 'MA': 5, 'KO': 4, 'NKE':
 4, 'F': 3, 'MCD': 3, 'AAPL': 2, 'CVX': 2, 'FDX': 2, 'T': 2, 'ABBV': 1, 'ABT': 1,
 'ACN': 1, 'AIG': 1, 'ALL': 1, 'AMGN': 1, 'AMZN': 1, 'APA': 1, 'APC': 1, 'AXP':
 1, 'BA': 1, 'BAC': 1, 'BIIB': 1, 'BK': 1, 'BMY': 1, 'C': 1, 'CAT': 1, 'CL': 1,
 'CMCSA': 1, 'COF': 1, 'COP': 1, 'COST': 1, 'CSCO': 1, 'CVS': 1, 'DD': 1, 'DIS':
 1, 'DOW': 1, 'DVN': 1, 'EMC': 1, 'EMR': 1, 'EXC': 1, 'FB': 1, 'FCX': 1, 'FOXA':
 1, 'GD': 1, 'GE': 1, 'GILD': 1, 'GM': 1, 'GOOG': 1, 'GS': 1, 'HAL': 1, 'HD': 1,
 'HON': 1, 'HPQ': 1, 'INTC': 1, 'JNJ': 1, 'JPM': 1, 'LLY': 1, 'LMT': 1, 'LOW': 1,
 'MDLZ': 1, 'MDT': 1, 'MET': 1, 'MMM': 1, 'MO': 1, 'MON': 1, 'MRK': 1, 'MS': 1,
 'MSFT': 1, 'NOV': 1, 'NSC': 1, 'ORCL': 1, 'OXY': 1, 'PEP': 1, 'PFE': 1, 'PG': 1,
 'QCOM': 1, 'RTN': 1, 'SBUX': 1, 'SLB': 1, 'SO': 1, 'SPG': 1, 'TGT': 1, 'TWX': 1,
 'TXN': 1, 'UNH': 1, 'UNP': 1, 'UPS': 1, 'USB': 1, 'UTX': 1, 'V': 1, 'VZ': 1,
 'XOM': 1}

```
{'F': 94, 'MA': 3, 'AAPL': 1, 'ABBV': 1, 'ABT': 1, 'ACN': 1, 'AIG': 1, 'ALL': 1,
'AMGN': 1, 'AMZN': 1, 'APA': 1, 'APC': 1, 'AXP': 1, 'BA': 1, 'BAC': 1, 'BAX': 1,
'BIIB': 1, 'BK': 1, 'BMY': 1, 'C': 1, 'CAT': 1, 'CL': 1, 'CMCSA': 1, 'COF': 1,
'COP': 1, 'COST': 1, 'CSCO': 1, 'CVS': 1, 'CVX': 1, 'DD': 1, 'DIS': 1, 'DOW': 1,
'DVN': 1, 'EBAY': 1, 'EMC': 1, 'EMR': 1, 'EXC': 1, 'FB': 1, 'FCX': 1, 'FDX': 1,
'FOXA': 1, 'GD': 1, 'GE': 1, 'GILD': 1, 'GM': 1, 'GOOG': 1, 'GS': 1, 'HAL': 1,
'HD': 1, 'HON': 1, 'HPQ': 1, 'IBM': 1, 'INTC': 1, 'JNJ': 1, 'JPM': 1, 'KO': 1,
'LLY': 1, 'LMT': 1, 'LOW': 1, 'MCD': 1, 'MDLZ': 1, 'MDT': 1, 'MET': 1, 'MMM': 1,
'MO': 1, 'MON': 1, 'MRK': 1, 'MS': 1, 'MSFT': 1, 'NKE': 1, 'NOV': 1, 'NSC': 1,
'ORCL': 1, 'OXY': 1, 'PEP': 1, 'PFE': 1, 'PG': 1, 'PM': 1, 'QCOM': 1, 'RTN': 1,
'SBUX': 1, 'SLB': 1, 'SO': 1, 'SPG': 1, 'T': 1, 'TGT': 1, 'TWX': 1, 'TXN': 1,
'UNH': 1, 'UNP': 1, 'UPS': 1, 'USB': 1, 'UTX': 1, 'V': 1, 'VZ': 1, 'WMT': 1,
'XOM': 1}
```

Yes they are relatively similar. But we can see that on both plots (hour and minute) the OneFactor has a more extended tail than the empiric one. Also the Gaussian is shorter than the other ones. However they have the same structure!

Creating Results: jupyter nbconvert Assignment_03.ipynb --to pdf --output
Assignment_03_Results-T2.pdf

Assignment_03

December 4, 2023

1 Authors

- Raphael Wäspi (18-918-938)
- Nicolas Huber (16-936-205)

```
[316]: import math
# Imports
import copy
import numpy as np
import matplotlib.pyplot as plt
import networkx as nx
import os
import glob
import re
import random
from matplotlib.ticker import FuncFormatter
from statistics import mean

import scipy.special
from tqdm import tqdm
import shutil
from joblib import Parallel, delayed
from scipy.special import factorial
import powerlaw
from NEMtropy import UndirectedGraph
from matplotlib.colors import LinearSegmentedColormap

[317]: # Define colors with reduced opacity from a colormap
COLOR_BLUE = 'dodgerblue'
COLOR_RED = 'lightcoral'
COLOR_GREY = 'slategrey'
COLOR_BLACK = 'black'
COLOR_LIGHT_GRAY = 'lightgray'
COLOR_DARK_GRAY = 'darkgray'

# Define the main data directory
OUTPUT_PATH = "./output"
```

```
[318]: # Prepare output folder

# Delete the entire 'output' directory and its contents
if os.path.exists(OUTPUT_PATH):
    shutil.rmtree(OUTPUT_PATH)

# Create the 'output' directory
os.makedirs(OUTPUT_PATH)

# Define subdirectories
OUTPUT_PATH_ER = os.path.join(OUTPUT_PATH, "erdos_renyi_avg_clustering")
OUTPUT_PATH_SYNTHETIC = os.path.join(OUTPUT_PATH, "synthetic")
OUTPUT_PATH_NODE_REMOVALS = os.path.join(OUTPUT_PATH, "node_removals")

# Recreate the subdirectories
os.makedirs(OUTPUT_PATH_ER)
os.makedirs(OUTPUT_PATH_SYNTHETIC)
os.makedirs(OUTPUT_PATH_NODE_REMOVALS)

[319]: # Define the directory containing the .gml files
directory = 'assignment_03_data'

# Use glob to get a list of all .gml files in the directory
gml_files = glob.glob(os.path.join(directory, '*.gml'))

# Init an empty dictionary for all graphs - the key is the actual name, and the
↳ value is the graph data
graphs = {}

# Use tqdm to show a progress bar in Jupyter Notebook
with tqdm(total=len(gml_files), desc="Loading GML Files") as pbar:
    # Loop through the list of .gml files and load them
    for file_path in gml_files:
        with open(file_path, 'r') as file:
            content = file.read()

            # Use regex to extract the desired part
            match = re.search(r'/(~/)+\.gml$', file_path)
            if match:
                graph_name = match.group(1)

                # Add to the graph dictionary
                graph = nx.read_gml(file_path)
                graphs[graph_name] = graph
```

```

        # Print the graph name when each part is finished
        print(f"Finished loading {graph_name}")

pbar.update(1) # Update the progress bar

```

Loading GML Files: 50%| | 1/2 [00:00<00:00, 6.35it/s]

Finished loading graph_eu_airlines

Loading GML Files: 100%| | 2/2 [00:00<00:00, 2.99it/s]

Finished loading graph_power

```

[320]: # Define the directory containing the .gml files
directory = 'assignment_03_data/World_Trade_Web'

# Use glob to get a list of all .gml files in the directory
gml_files = glob.glob(os.path.join(directory, '*.graphml'))

# Init an empty dictionary for all graphs - the key is the actual name, and the
# value is the graph data
graphs_wtn = {}

# Use tqdm to show a progress bar in Jupyter Notebook
with tqdm(total=len(gml_files), desc="Loading GML WTW Files") as pbar:
    # Loop through the list of .gml files and load them
    for file_path in gml_files:
        with open(file_path, 'r') as file:
            content = file.read()

            # Use regex to extract the desired part
            match = re.search(r'/(~/+)\.txt.graphml$', file_path)
            if match:
                graph_name = match.group(1)

                # Add to the graph dictionary
                graph = nx.read_graphml(file_path)
                graphs_wtn[graph_name] = graph

                # Print the graph name when each part is finished
                print(f"Finished loading {graph_name}")

pbar.update(1) # Update the progress bar

```

Loading GML WTW Files: 9%| | 1/11 [00:00<00:01, 5.05it/s]

Finished loading WDN_1997

Finished loading WDN_1992


```

Loading GML WTW Files: 36%|          | 4/11 [00:00<00:01, 6.39it/s]
Finished loading WDN_2000
Finished loading WDN_1998

Loading GML WTW Files: 45%|          | 5/11 [00:00<00:00, 6.96it/s]
Finished loading WDN_1994
Finished loading WDN_1993

Loading GML WTW Files: 73%|          | 8/11 [00:03<00:01, 1.54it/s]
Finished loading WDN_2002
Finished loading WDN_1996

Loading GML WTW Files: 91%|          | 10/11 [00:04<00:00, 2.35it/s]
Finished loading WDN_1995
Finished loading WDN_1999

Loading GML WTW Files: 100%|         | 11/11 [00:04<00:00, 2.42it/s]
Finished loading WDN_2001

```

2 III Robustness

2.1 1. Generate synthetic networks (you can use whatever module you prefer):

1. Random networks (ErdosRényi) (with $n = 1000$ and $\langle k \rangle = \{1, 4\}$)
2. Scale-free networks (Barabási-Albert) (with $n = 1000$ and $m = \{2, 4\}$)

```

[321]: N = 1000
K_VALUES = [1, 4] # range(1, 5, 1)
M_VALUES = [2, 4] # range(2, 5, 1)
CONSIDERED_GRAPHs = ['graph_eu_airlines', 'graph_power']

```

```

[322]: er_graphs = {}
ba_graphs = {}

```

```

[323]: def generate_ER_graphs(nodes, k_values, number):
    graphs = {}

    # Generate n graphs for each k value
    for avg_degree in k_values:
        graphs[avg_degree] = {}

        for index in range(1, number+1, 1):

            probability = avg_degree / (nodes - 1)

            graph = nx.erdos_renyi_graph(n=nodes, p=probability)

```

```

        graphs[avg_degree][index] = graph

    return graphs

```

```

[324]: er_graphs = generate_ER_graphs(nodes=N, k_values=K_VALUES, number=1)
er_graphs

```

```

[324]: {1: {1: <networkx.classes.graph.Graph at 0x11f733fd0>},
      4: {1: <networkx.classes.graph.Graph at 0x155c32210>}}

```

```

[325]: def generate_BA_graphs(nodes, m_values, number):
    graphs = {}

    # Generate n graphs for each m value
    for m in m_values:
        graphs[m] = {}

        for index in range(1, number+1, 1):

            graph = nx.barabasi_albert_graph(n=nodes, m=m)

            graphs[m][index] = graph

    return graphs

```

```

[326]: ba_graphs = generate_BA_graphs(nodes=N, m_values=M_VALUES, number=1)
ba_graphs

```

```

[326]: {2: {1: <networkx.classes.graph.Graph at 0x11fad7ed0>},
      4: {1: <networkx.classes.graph.Graph at 0x1201531d0>}}

```

2.2 2. Plot the degree distributions of the synthetic networks and of the networks provided in the datasets.

```

[327]: def plot_degree_distribution(ax, graph, color, param, graph_index, er):
    degrees = [degree for node, degree in graph.degree()]
    degree_freq = nx.degree_histogram(graph)
    ax.bar(range(len(degree_freq)), degree_freq, width=0.8, color=color)
    ax.set_xlabel('Degree')
    ax.set_ylabel('Frequency')
    if er:
        ax.set_title(fr'Degree Distribution of graph iteration {graph_index}
↪with  $\langle k \rangle = \{param\}$ ')
    else:
        ax.set_title(fr'Degree Distribution of graph iteration {graph_index}
↪with  $m = \{param\}$ ')

```

```

[328]: # ER-graphs
# Define colors for different average degrees using a colormap
num_avg_degrees = len(er_graphs)
colors = plt.cm.viridis_r([i / num_avg_degrees for i in range(num_avg_degrees)])

# Create a subplot grid
num_instances = len(next(iter(er_graphs.values())))) # Assuming all instances
↳ have the same count
fig, axs = plt.subplots(num_instances, num_avg_degrees, figsize=(20, 8))

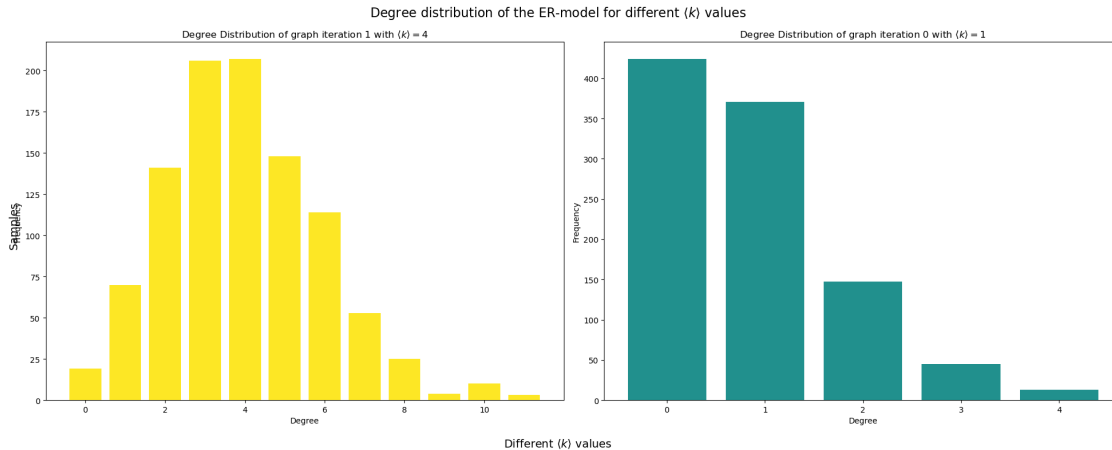
# Iterate through avg_degree_index and graph_index
for avg_degree_index, (avg_degree, avg_degree_plots) in enumerate(er_graphs.
↳ items()):
    color = colors[avg_degree_index - 1] # Get color for the degree
    for graph_index, graph in avg_degree_plots.items():
        # Get the appropriate axis for the subplot
        if num_instances == 1:
            ax = axs[avg_degree_index - 1]
        else:
            ax = axs[graph_index - 1, avg_degree_index - 1]

        # Plot on the corresponding axis with the assigned color
        plot_degree_distribution(ax=ax, graph=graph, color=color,
↳ param=avg_degree, graph_index=avg_degree_index, er=True)

fig.suptitle(fr"Degree distribution of the ER-model for different $\langle k \rangle$
↳ $\langle k \rangle$ values", fontsize=16)
fig.supxlabel(fr"Different $\langle k \rangle$ values", fontsize=14)
fig.supylabel("Samples", fontsize=14)

# Adjust layout and display the plot
plt.tight_layout()
file_name=f"degree_dist_er_synth_networks.png"
complete_plot_filename=os.path.join(OUTPUT_PATH_SYNTHETIC, file_name)
plt.savefig(complete_plot_filename)
plt.show()

```



```
[329]: # BA-graphs
# Define colors for different average degrees using a colormap
num_ba_values = len(ba_graphs)
colors = plt.cm.viridis_r([i / num_ba_values for i in range(num_ba_values)])
# Create a subplot grid
num_instances = len(next(iter(ba_graphs.values()))) # Assuming all instances
↳ have the same count
fig, axs = plt.subplots(num_instances, num_ba_values, figsize=(20, 8))

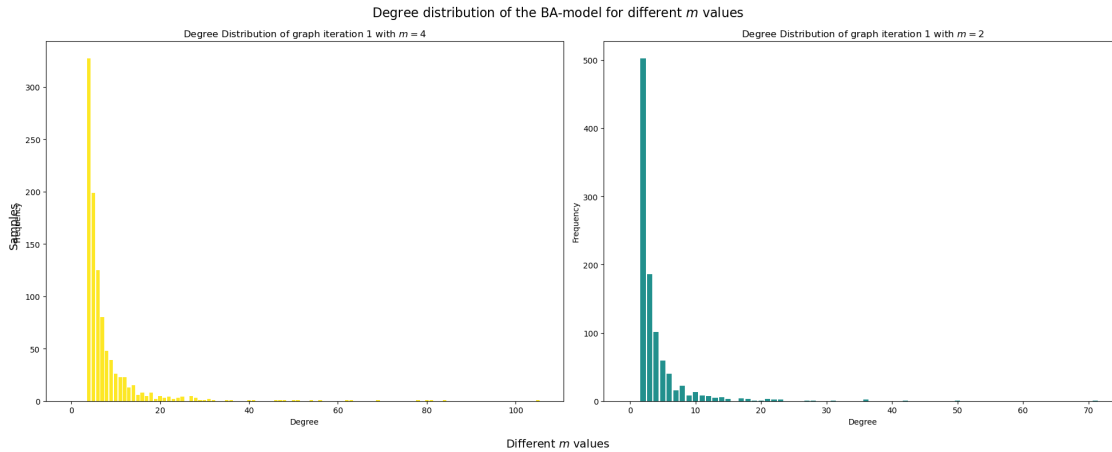
# Iterate through avg_degree_index and graph_index
for m_index, (m, m_plots) in enumerate(ba_graphs.items()): # Do not forget that
↳ m is not an index -> use enumerate and m_index
    color = colors[m_index - 1] # Get color for the degree
    for graph_index, graph in m_plots.items():
        # Get the appropriate axis for the subplot
        if num_instances == 1:
            ax = axs[m_index - 1]
        else:
            ax = axs[graph_index - 1, m_index - 1]

        # Plot on the corresponding axis with the assigned color
        plot_degree_distribution(ax=ax, graph=graph, color=color, param=m,
↳ graph_index=graph_index, er=False)

fig.suptitle(fr"Degree distribution of the BA-model for different $m$ values",
↳ fontsize=16)
fig.supxlabel(fr"Different $m$ values", fontsize=14)
fig.supylabel("Samples", fontsize=14)

# Adjust layout and display the plot
```

```
plt.tight_layout()
file_name=f"degree_dist_ba_synth_networks.png"
complete_plot_filename=os.path.join(OUTPUT_PATH_SYNTHETIC, file_name)
plt.savefig(complete_plot_filename)
plt.show()
```



2.3 3. Plot the size of the largest connected component as a function of removed edges and nodes in two scenarios for the synthetic graphs and the real networks in the datasets:

- (a) Random attacks (failures)
- (b) Targeted attacks (degree centrality and betweenness centrality)

```
[330]: K_ESTIMATOR = 100
```

```
[331]: def remove_random_nodes(graph, n):
    # Copy it such that we do not need to import it again
    updated_graph = copy.deepcopy(graph)
    nodes_list = list(updated_graph.nodes())

    if len(nodes_list) < n:
        return graph, []

    extracted_nodes = []
    for _ in range(n):
        if nodes_list:
            random_node = random.choice(nodes_list)
            updated_graph.remove_node(random_node)
            extracted_nodes.append(random_node)
            nodes_list.remove(random_node)
        else:
```

```

        break

    # DEBUG
    # print(f"Removed nodes: {extracted_nodes}")
    return updated_graph, extracted_nodes

```

```

[332]: def remove_random_percentage_nodes(graph, percentage):
        if not 0 <= percentage <= 100:
            raise ValueError("Percentage should be between 0 and 100")

        num_nodes = len(graph.nodes())
        num_to_remove = int(num_nodes * (percentage / 100))

        return remove_random_nodes(graph, num_to_remove)

```

```

[333]: PERCENTAGES_TO_BE_REMOVED = range(1, 100, 1)
        #PERCENTAGES_TO_BE_REMOVED = [1,50]

```

```

[334]: TOP_N_TO_BE_REMOVED = [0,1,2,3,4,5,10]
        #TOP_N_TO_BE_REMOVED = [1,2]

```

```

[335]: random_removed_graphs = {}

```

```

[336]: def generate_random_removed_graphs(graphs, percentages_to_be_removed,
        ↪top_n_to_be_removed):
        random_removed_graphs = {}

        def process_graph(graph, graph_name, action, value):
            nonlocal random_removed_graphs
            if graph_name not in random_removed_graphs:
                random_removed_graphs[graph_name] = {}

            random_removed_graphs[graph_name][action] = {}
            updated_graph, extracted_nodes = value
            random_removed_graphs[graph_name][action]["graph"] = updated_graph
            removed_nodes = graph.number_of_nodes() - updated_graph.
            ↪number_of_nodes()
            random_removed_graphs[graph_name][action]["reduced_nodes"] =
            ↪removed_nodes
            #print(f"{action} of graph {graph_name}")
            #print(f"Graph {graph_name} has {graph.number_of_nodes()}")
            #print(f"Graph (updated) {graph_name} has {updated_graph.
            ↪number_of_nodes()}")
            #print(f"Removed Nodes: {removed_nodes}")
            #print("---")

        for graph_name, graph in graphs.items():

```

```

        for percentage in percentages_to_be_removed:
            action = str(permission) + '%'
            process_graph(graph, graph_name, action,
                remove_random_percentage_nodes(graph=graph, percentage=percentage))

        for top_n in top_n_to_be_removed:
            action = f"{top_n}"
            process_graph(graph, graph_name, action,
                remove_random_nodes(graph=graph, n=top_n))

    return random_removed_graphs

```

```

[337]: random_removed_graphs = generate_random_removed_graphs(graphs=graphs,
    remove_random_percentage_nodes=PERCENTAGES_TO_BE_REMOVED,
    remove_random_nodes=TOP_N_TO_BE_REMOVED)

```

```

[338]: def sort_nodes_by_degree centrality(graph):
    degree centrality = nx.degree centrality(graph)
    sorted_by_degree = sorted(degree centrality.items(), key=lambda x: x[1],
        reverse=True)
    return sorted_by_degree

```

```

[339]: def sort_nodes_by_betweenness centrality(graph):
    betweenness centrality = nx.betweenness centrality(graph)
    sorted_by_betweenness = sorted(betweenness centrality.items(), key=lambda x:
        x[1], reverse=True)
    return sorted_by_betweenness

```

```

[340]: def remove_targeted_nodes(graph, centrality_measure, n):
    if centrality_measure == 'degree':
        sorted_nodes = sorted(nx.degree centrality(graph).items(), key=lambda x:
            x[1], reverse=True)
    elif centrality_measure == 'betweenness':
        sorted_nodes = sorted(nx.betweenness centrality(graph, k=K_ESTIMATOR).
            items(), key=lambda x: x[1], reverse=True)
    else:
        raise ValueError("Centrality measure should be 'degree' or
            'betweenness'")

    updated_graph = copy.deepcopy(graph)

    if len(sorted_nodes) < n:
        return graph, []

    extracted_nodes = []
    for i in range(n):
        node = sorted_nodes[i][0] # Extracting node from the sorted list

```

```

        updated_graph.remove_node(node)
        extracted_nodes.append(node)

    # DEBUG
    # print(f"Removed nodes: {extracted_nodes}")
    return updated_graph, extracted_nodes

```

```

[341]: def remove_target_percentage_nodes(graph, centrality_measure, percentage):
        if not 0 <= percentage <= 100:
            raise ValueError("Percentage should be between 0 and 100")

        num_nodes = len(graph.nodes())
        num_to_remove = int(num_nodes * (percentage / 100))

        return remove_targeted_nodes(graph, centrality_measure, num_to_remove)

```

```

[342]: target_degree Centrality_removed_graphs = {}

```

```

[343]: def generate_target_degree_Centrality_removed_graphs(graphs,
    ↪percentages_to_be_removed, top_n_to_be_removed):
        target_degree Centrality_removed_graphs = {}

        def process_graph(graph, graph_name, action, value):
            nonlocal target_degree Centrality_removed_graphs
            if graph_name not in target_degree Centrality_removed_graphs:
                target_degree Centrality_removed_graphs[graph_name] = {}

            target_degree Centrality_removed_graphs[graph_name][action] = {}
            updated_graph, extracted_nodes = value
            target_degree Centrality_removed_graphs[graph_name][action]["graph"] =
    ↪updated_graph
            removed_nodes = graph.number_of_nodes() - updated_graph.
    ↪number_of_nodes()

            ↵
            ↪target_degree Centrality_removed_graphs[graph_name][action]["reduced_nodes"]
    ↪= removed_nodes

            #print(f"{action} of graph {graph_name}")
            #print(f"Graph {graph_name} has {graph.number_of_nodes()}")
            #print(f"Graph (updated) {graph_name} has {updated_graph.
    ↪number_of_nodes()}")
            #print(f"Removed Nodes: {removed_nodes}")
            #print("---")

        for graph_name, graph in graphs.items():
            for percentage in percentages_to_be_removed:
                action = str(percentage) + '%'

```



```

        process_graph(graph, graph_name, action,
↳remove_target_percentage_nodes(graph=graph, centrality_measure='degree',
↳percentage=percentage))

    for top_n in top_n_to_be_removed:
        action = f"{top_n}"
        process_graph(graph, graph_name, action,
↳remove_targeted_nodes(graph=graph, centrality_measure='degree', n=top_n))

    return target_degree Centrality_removed_graphs

```

```

[344]: target_degree Centrality_removed_graphs =
↳generate_target_degree Centrality_removed_graphs(graphs=graphs,
↳percentages_to_be_removed=PERCENTAGES_TO_BE_REMOVED,
↳top_n_to_be_removed=TOP_N_TO_BE_REMOVED)

```

```

[345]: target_betweenness Centrality_removed_graphs = {}

```

```

[346]: def generate_target_betweenness Centrality_removed_graphs(graphs,
↳percentages_to_be_removed, top_n_to_be_removed):
    target_betweenness Centrality_removed_graphs = {}

    def process_graph(graph, graph_name, action, value):
        nonlocal target_betweenness Centrality_removed_graphs
        if graph_name not in target_betweenness Centrality_removed_graphs:
            target_betweenness Centrality_removed_graphs[graph_name] = {}

            target_betweenness Centrality_removed_graphs[graph_name][action] = {}
            updated_graph, extracted_nodes = value

            ↳
            ↳target_betweenness Centrality_removed_graphs[graph_name][action]["graph"] =
            ↳updated_graph
                removed_nodes = graph.number_of_nodes() - updated_graph.
            ↳number_of_nodes()

            ↳
            ↳target_betweenness Centrality_removed_graphs[graph_name][action]["reduced_nodes"]
            ↳= removed_nodes

                #print(f"{action} of graph {graph_name}")
                #print(f"Graph {graph_name} has {graph.number_of_nodes()}")
                #print(f"Graph (updated) {graph_name} has {updated_graph.
            ↳number_of_nodes()}")
                #print(f"Removed Nodes: {removed_nodes}")
                #print("---")

    for graph_name, graph in graphs.items():
        for percentage in percentages_to_be_removed:
            action = str(percentage) + '%'

```

```

        process_graph(graph, graph_name, action,
        ↪remove_target_percentage_nodes(graph=graph,
        ↪centrality_measure='betweenness', percentage=percentage))

        for top_n in top_n_to_be_removed:
            action = f"{top_n}"
            process_graph(graph, graph_name, action,
            ↪remove_targeted_nodes(graph=graph, centrality_measure='betweenness',
            ↪n=top_n))

        return target_betweenness_centrality_removed_graphs

```

```

[347]: target_betweenness_centrality_removed_graphs =
        ↪generate_target_betweenness_centrality_removed_graphs(graphs=graphs,
        ↪percentages_to_be_removed=PERCENTAGES_TO_BE_REMOVED,
        ↪top_n_to_be_removed=TOP_N_TO_BE_REMOVED)

```

```

[348]: #random_removed_graphs

```

```

[349]: plot_data_random = {}

```

```

[350]: def generate_plot_data(graphs):
        plot_data = {}

        for graph_name, cases in graphs.items():
            #print(f"Graph: {graph_name}")
            plot_data[graph_name] = {
                'percentage': {'x_percentages_removed': [], 'y_size_lcc': [],
                ↪'y_size_lcc_frac': []},
                'absolute': {'x_absolute_removed': [], 'y_size_lcc': [],
                ↪'y_size_lcc_frac': []}
            }

            for case, data in cases.items():
                graph = data['graph']
                largest_cc = max(nx.connected_components(graph), key=len)
                largest_connected_component = graph.subgraph(largest_cc)
                size_largest_cc = len(largest_connected_component)

                #print(f"Case: {case}")
                #print(f"Size of largest connected component: {size_largest_cc}")
                if '%' in str(case):
                    plot_data[graph_name]['percentage']['x_percentages_removed'].
                    ↪append(int(str(case).replace('%', ''))/100)
                    plot_data[graph_name]['percentage']['y_size_lcc'].
                    ↪append(size_largest_cc)

```

```

        plot_data[graph_name]['percentage']['y_size_lcc_frac'].
↪append(size_largest_cc/plot_data[graph_name]['percentage']['y_size_lcc'][0])
    else:
        plot_data[graph_name]['absolute']['x_absolute_removed'].
↪append(case)
        plot_data[graph_name]['absolute']['y_size_lcc'].
↪append(size_largest_cc)
        plot_data[graph_name]['absolute']['y_size_lcc_frac'].
↪append(size_largest_cc/plot_data[graph_name]['absolute']['y_size_lcc'][0])

    #print()

    return plot_data

```

```
[351]: plot_data_random = generate_plot_data(random_removed_graphs)
```

```
[352]: #plot_data_random
```

```
[353]: def percent_formatter(x, pos):
        return '{:.0%}'.format(x)
```

```
[354]: def lowercase_and_replace_spaces(input_string):
        # Convert the string to lowercase
        lowercase_string = input_string.lower()
        # Replace spaces with underscores
        modified_string = lowercase_string.replace(' ', '_')
        modified_string = modified_string.replace(':', '')
        return modified_string

def plot_removed_data(plot_data, title):
    fig, axs = plt.subplots(2, 2, figsize=(18, 18))
    for graph_name, data in plot_data.items():
        # x: percentage of removed nodes, y: absolute size of lcc after removal
        axs[0, 0].plot(data['percentage']['x_percentages_removed'],
                        data['percentage']['y_size_lcc'],
                        label=graph_name, marker='o')

        # x: absolute removed nodes, y: absolute size of lcc after removal
        axs[0, 1].plot(data['absolute']['x_absolute_removed'],
                        data['absolute']['y_size_lcc'],
                        label=graph_name, marker='o')

        # x: percentage of removed nodes, y: percentage of lcc after removal
        ↪(lcc / lcc_before_removal)
        axs[1, 0].plot(data['percentage']['x_percentages_removed'],
                        data['percentage']['y_size_lcc_frac'],
                        label=graph_name, marker='o')

```

```

        # x: absolute removed nodes, y: percentage of lcc after removal (lcc /  $\hookrightarrow$  lcc_before_removal)
        axs[1, 1].plot(data['absolute']['x_absolute_removed'],
                        data['absolute']['y_size_lcc_frac'],
                        label=graph_name, marker='o')

    # Set x-axis and y-axis labels for each subplot
    axs[0, 0].set_xlabel('% removed nodes')
    axs[0, 0].set_ylabel('# nodes of lcc')

    axs[0, 1].set_xlabel('# removed nodes')
    axs[0, 1].set_ylabel('# nodes of lcc')

    axs[1, 0].set_xlabel('% removed nodes')
    axs[1, 0].set_ylabel('relative size lcc after removal (%)')

    axs[1, 1].set_xlabel('# removed nodes')
    axs[1, 1].set_ylabel('relative size lcc after removal (%)')

    plt.suptitle(title, fontsize=16)

    axs[0, 0].axis.set_major_formatter(FuncFormatter(percent_formatter))
    axs[1, 0].axis.set_major_formatter(FuncFormatter(percent_formatter))
    axs[0, 0].set_title('Absolute')
    axs[0, 1].set_title('Absolute')
    axs[1, 0].set_title('Percentage')
    axs[1, 1].set_title('Percentage')

    # Get handles and labels for all scatter plots
    handles, labels = axs[0, 0].get_legend_handles_labels()

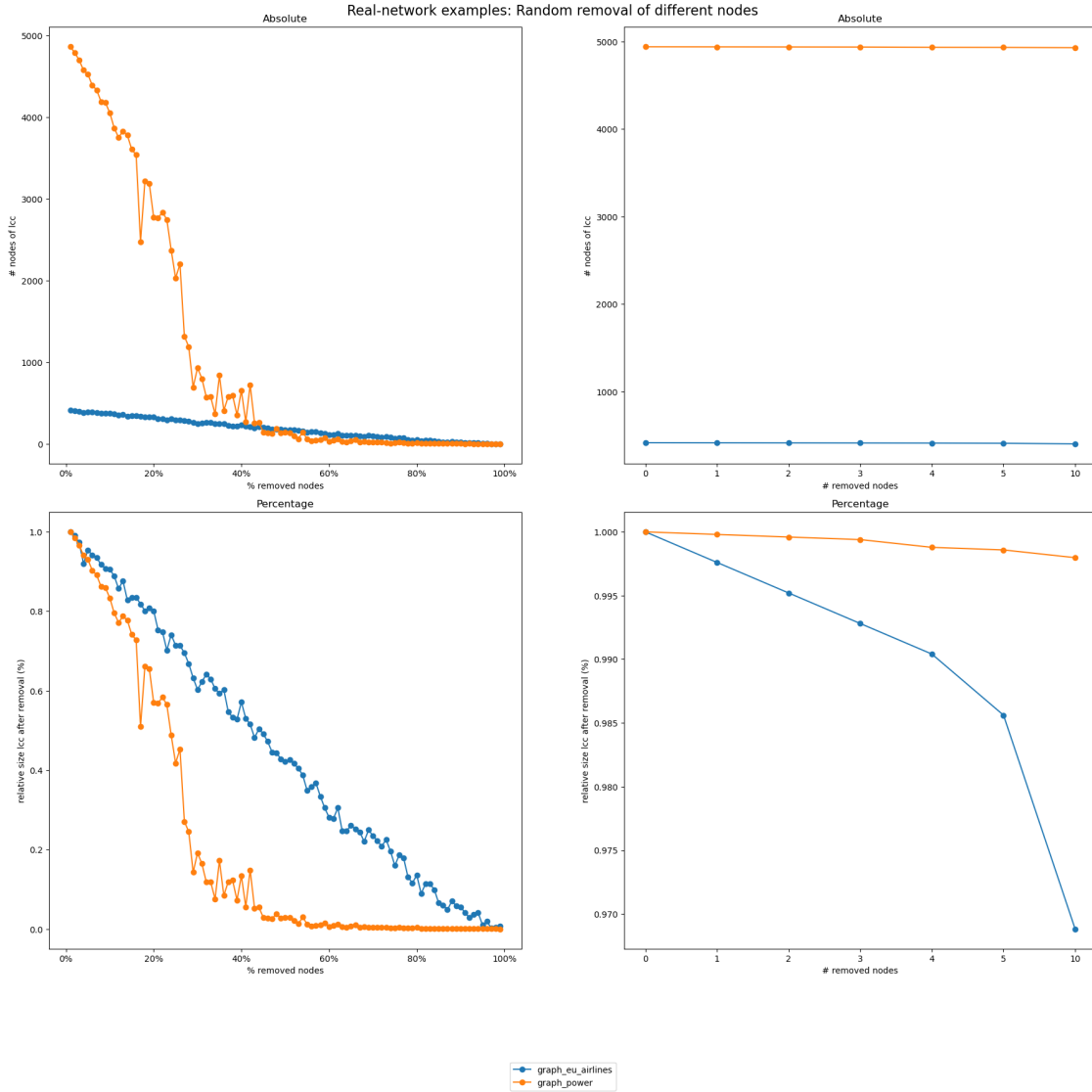
    # Create a legend for all scatter plots using handles and labels collected  $\hookrightarrow$ 
     $\hookrightarrow$ above
    plt.legend(handles, labels, loc='upper left', bbox_to_anchor=(-0.25, -0.25))
    plt.tight_layout()
    title_lc = lowercase_and_replace_spaces(title)
    file_name=f"{title_lc}.png"
    complete_plot_filename=os.path.join(OUTPUT_PATH_NODE_REMOVALS, file_name)
    plt.savefig(complete_plot_filename)
    plt.show()

```

```

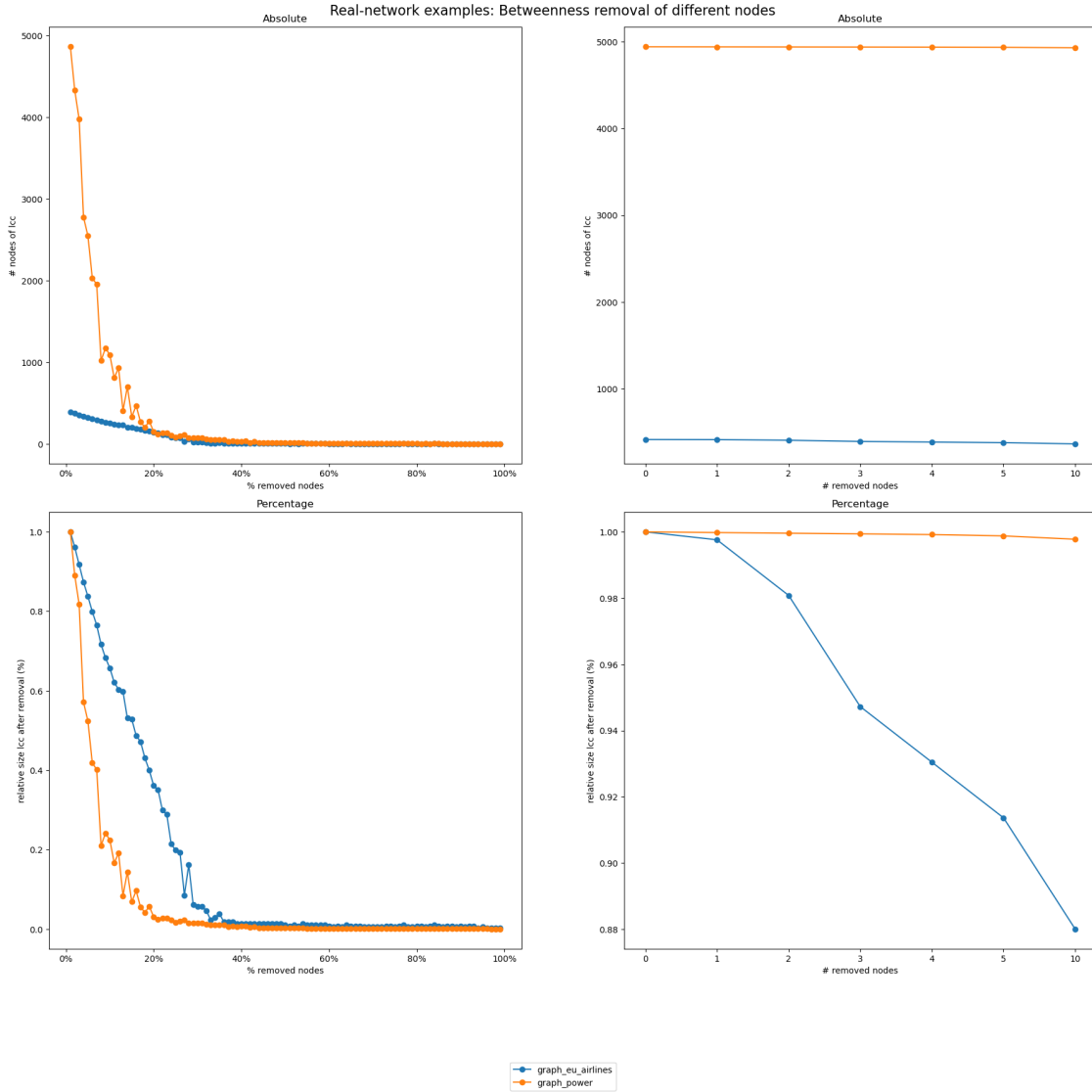
[355]: plot_removed_data(plot_data_random, 'Real-network examples: Random removal of  $\hookrightarrow$ 
 $\hookrightarrow$ different nodes')

```



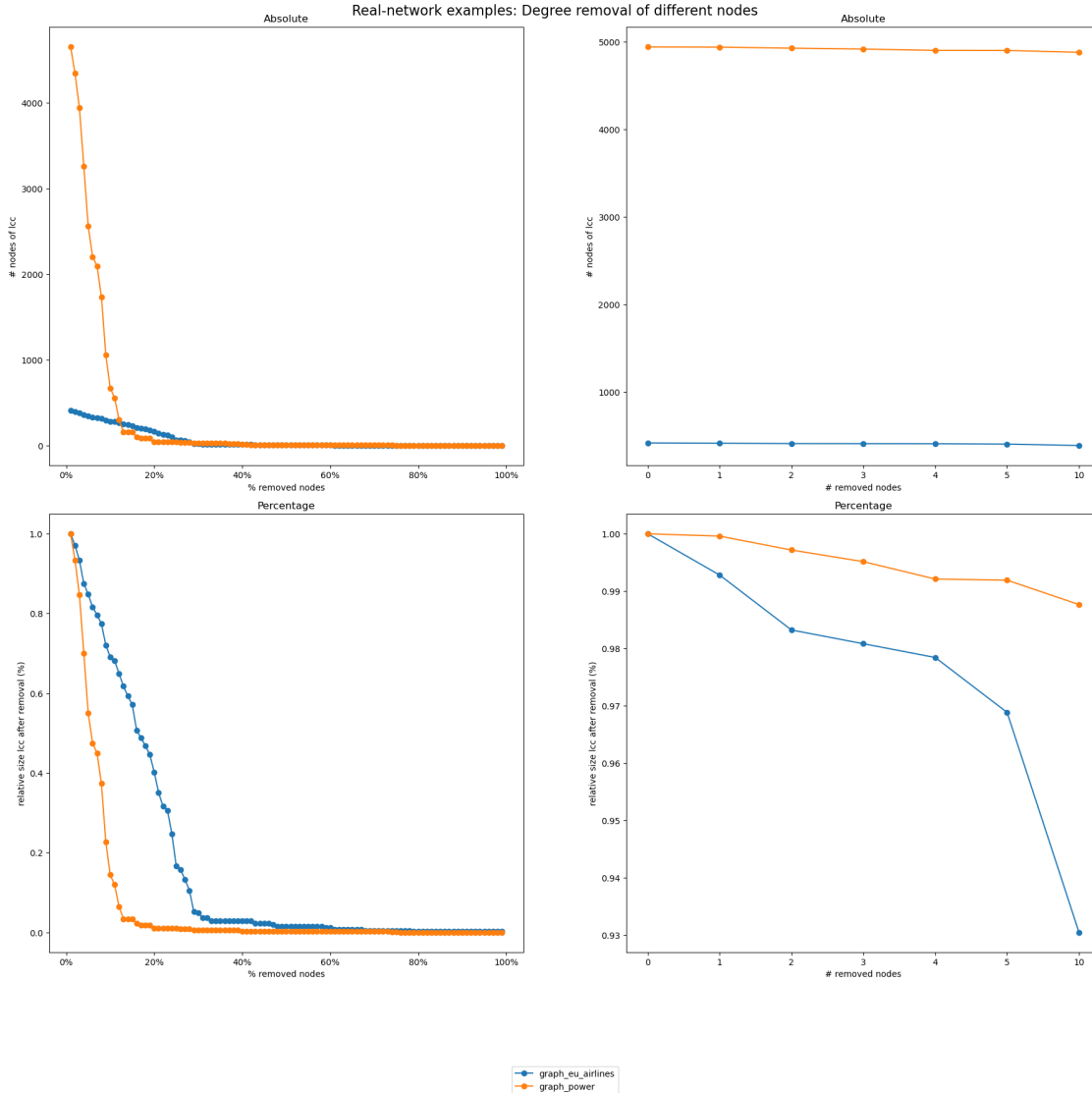
```
[356]: plot_data_betweenness = □
        ↳ generate_plot_data(target_betweenness_centrality_removed_graphs)
```

```
[357]: plot_removed_data(plot_data_betweenness, 'Real-network examples: Betweenness □
        ↳ removal of different nodes')
```



```
[358]: plot_data_degree = generate_plot_data(target_degree_centralty_removed_graphs)

[359]: plot_removed_data(plot_data_degree, 'Real-network examples: Degree removal of_
↪different nodes')
```



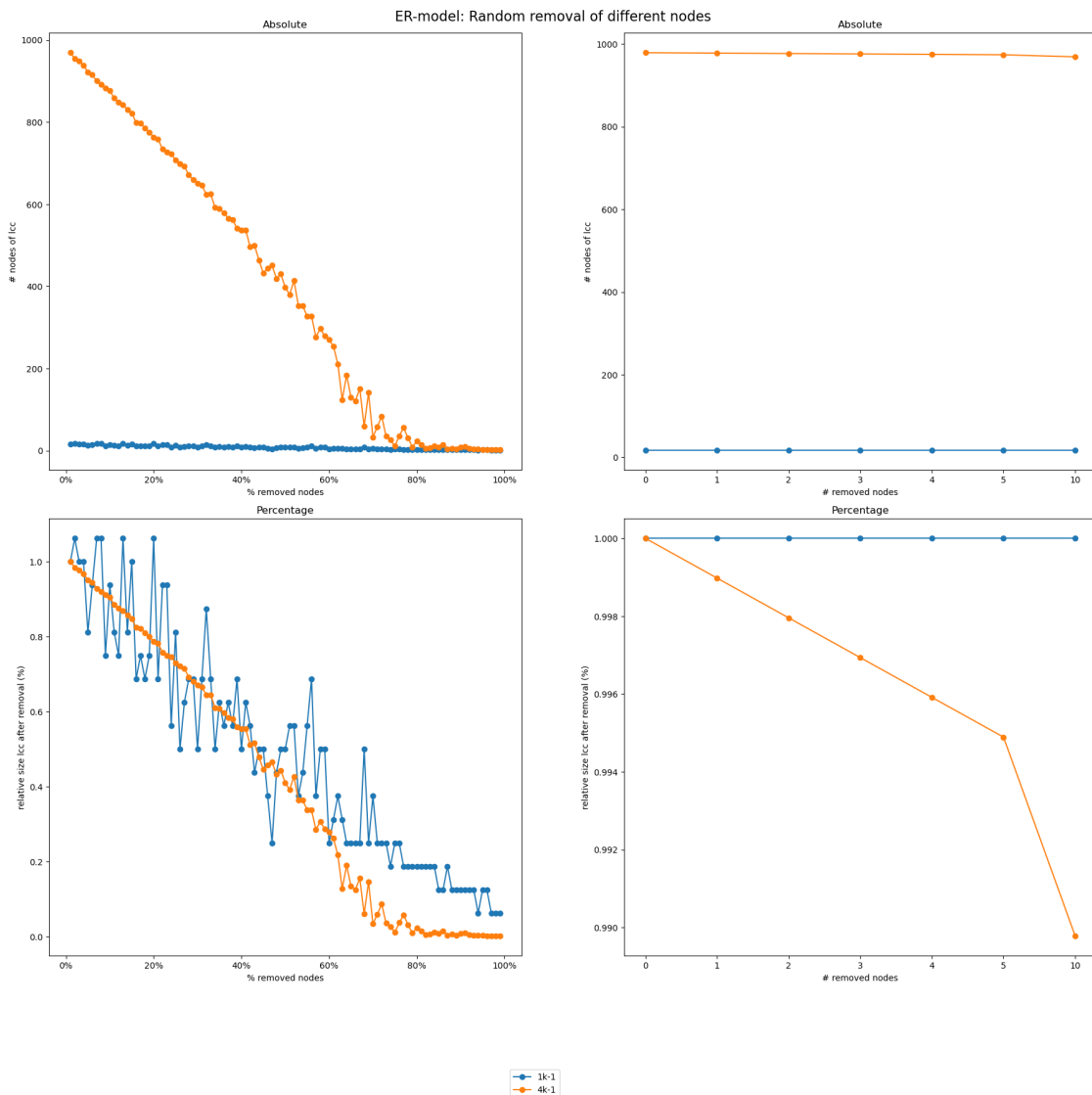
```
[360]: ba_graphs
```

```
[360]: {2: {1: <networkx.classes.graph.Graph at 0x11fad7ed0>},
      4: {1: <networkx.classes.graph.Graph at 0x1201531d0>}}
```

```
[361]: er_graphs_samples = {f"{outer_key}k-{inner_key}": value for outer_key,
    inner_dict in er_graphs.items() for inner_key, value in inner_dict.items()}
print(er_graphs_samples)
```

```
{'1k-1': <networkx.classes.graph.Graph object at 0x11f733fd0>, '4k-1':
<networkx.classes.graph.Graph object at 0x155c32210>}
```

```
[362]: random_removed_er_graphs =
↳generate_random_removed_graphs(graphs=er_graphs_samples,
↳percentages_to_be_removed=PERCENTAGES_TO_BE_REMOVED,
↳top_n_to_be_removed=TOP_N_TO_BE_REMOVED)
plot_er_data_random = generate_plot_data(random_removed_er_graphs)
plot_removed_data(plot_er_data_random, 'ER-model: Random removal of different
↳nodes')
```



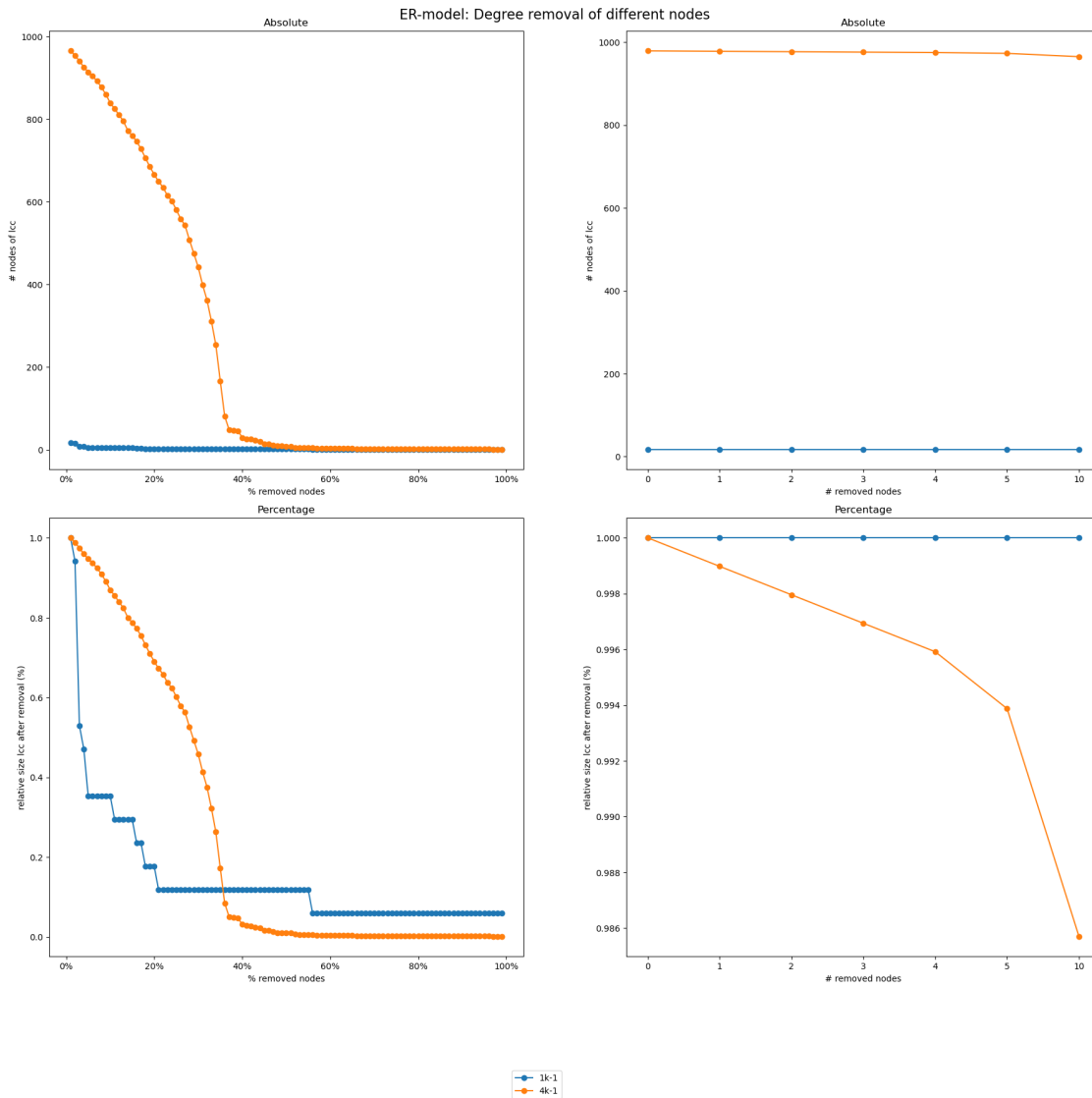
```
[363]: degree_removed_er_graphs =
↳generate_target_degree_centralty_removed_graphs(graphs=er_graphs_samples,
↳percentages_to_be_removed=PERCENTAGES_TO_BE_REMOVED,
↳top_n_to_be_removed=TOP_N_TO_BE_REMOVED)
```



```

plot_er_data_degree = generate_plot_data(degree_removed_er_graphs)
plot_removed_data(plot_er_data_degree, 'ER-model: Degree removal of different_
↳nodes')

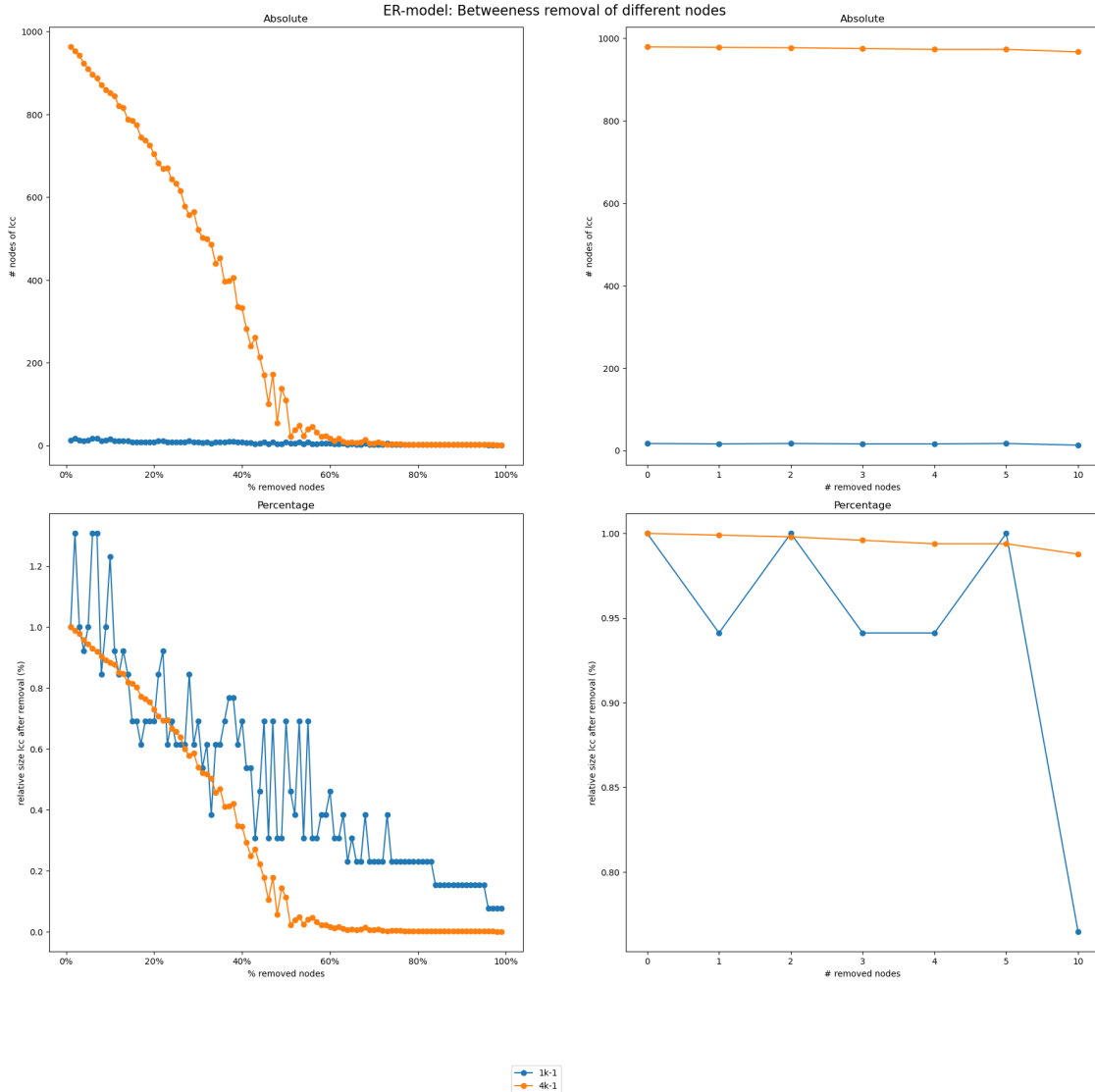
```



```

[364]: betweenness_removed_er_graphs =
↳generate_target_betweenness_centrality_removed_graphs(graphs=er_graphs_samples,
↳percentages_to_be_removed=PERCENTAGES_TO_BE_REMOVED,
↳top_n_to_be_removed=TOP_N_TO_BE_REMOVED)
plot_er_data_betweenness = generate_plot_data(betweenness_removed_er_graphs)
plot_removed_data(plot_er_data_betweenness, 'ER-model: Betweenness removal of_
↳different nodes')

```

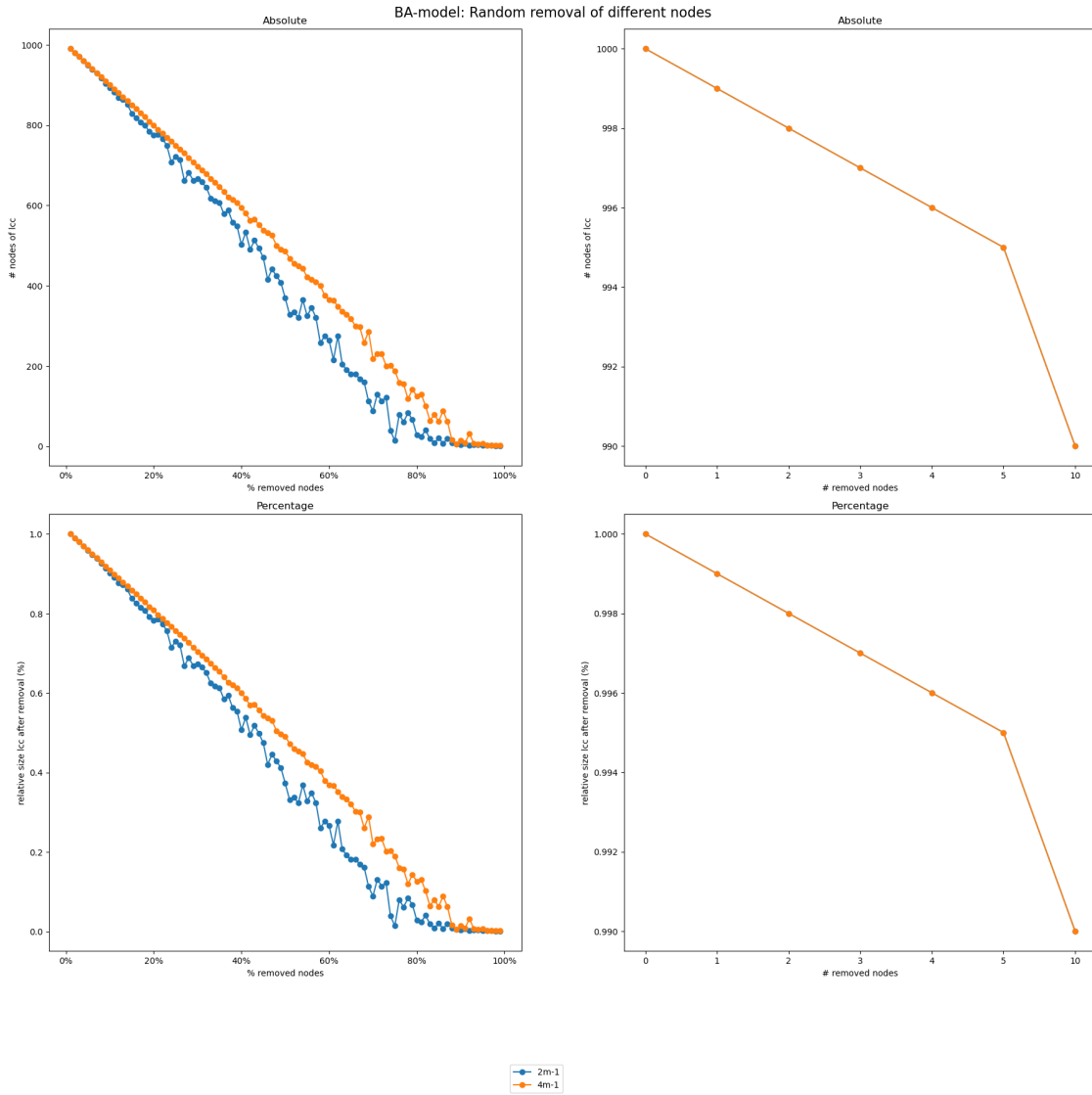


```
[365]: ba_graphs_samples = {f"{outer_key}m-{inner_key}": value for outer_key,
    ↳ inner_dict in ba_graphs.items() for inner_key, value in inner_dict.items()}
print(ba_graphs_samples)
```

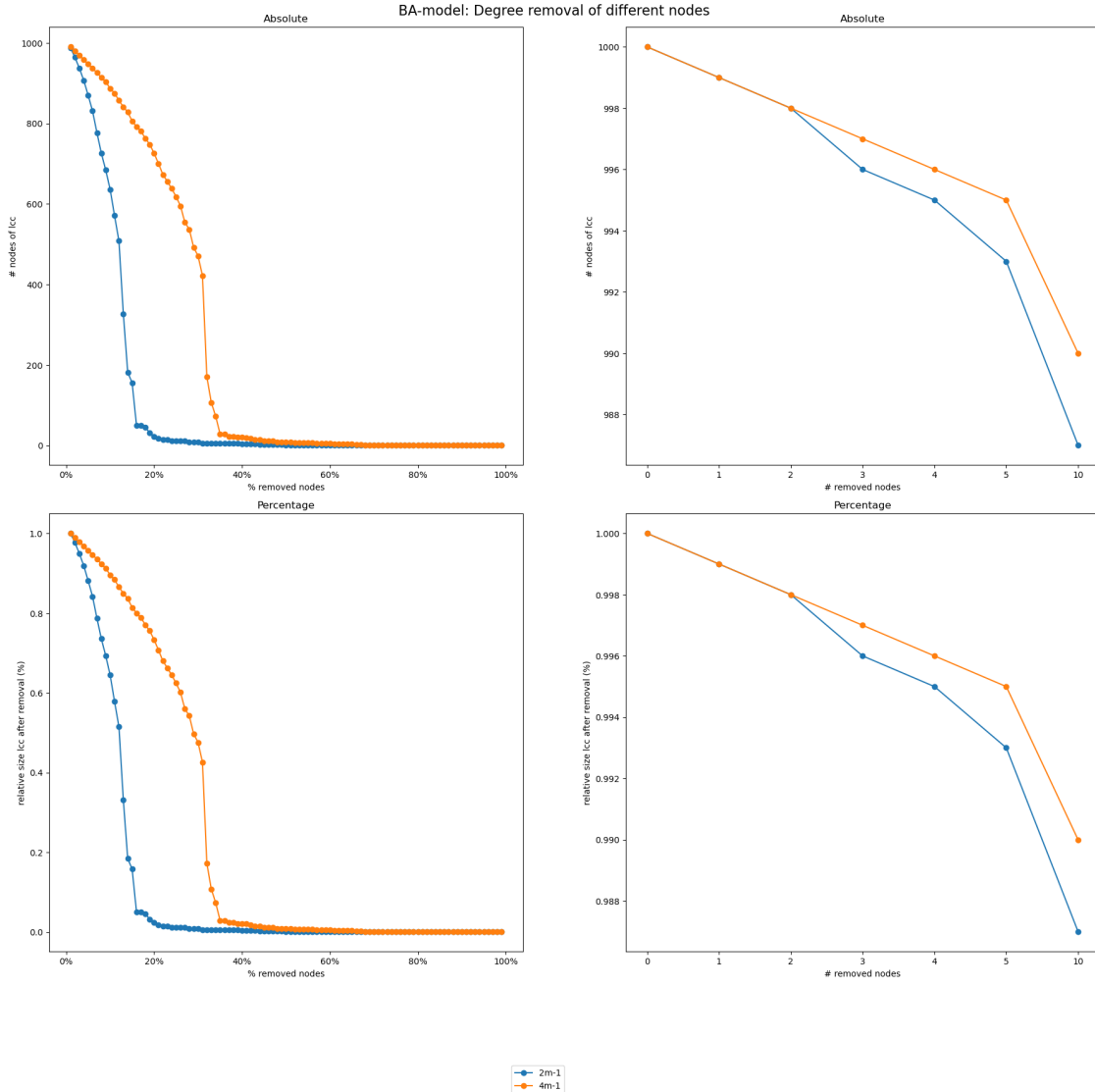
```
{'2m-1': <networkx.classes.graph.Graph object at 0x11fad7ed0>, '4m-1':
<networkx.classes.graph.Graph object at 0x1201531d0>}
```

```
[366]: random_removed_ba_graphs =
    ↳ generate_random_removed_graphs(graphs=ba_graphs_samples,
    ↳ percentages_to_be_removed=PERCENTAGES_TO_BE_REMOVED,
    ↳ top_n_to_be_removed=TOP_N_TO_BE_REMOVED)
plot_ba_data_random = generate_plot_data(random_removed_ba_graphs)
```

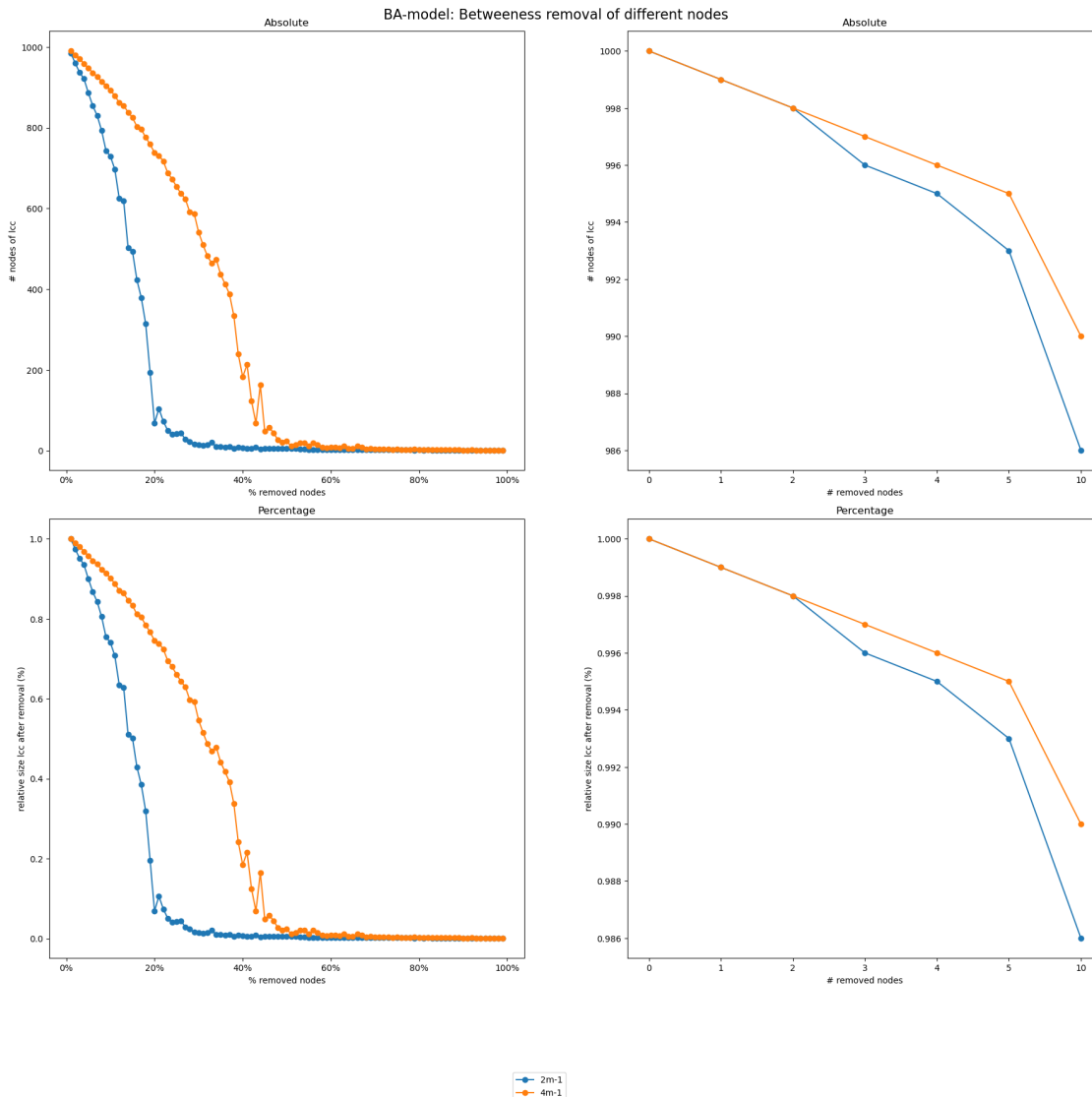
```
plot_removed_data(plot_ba_data_random, 'BA-model: Random removal of different_
↳nodes')
```



```
[367]: degree_removed_ba_graphs =_
↳generate_target_degree centrality_removed_graphs(graphs=ba_graphs_samples,_
↳percentages_to_be_removed=PERCENTAGES_TO_BE_REMOVED,_
↳top_n_to_be_removed=TOP_N_TO_BE_REMOVED)
plot_ba_data_degree = generate_plot_data(degree_removed_ba_graphs)
plot_removed_data(plot_ba_data_degree, 'BA-model: Degree removal of different_
↳nodes')
```



```
[368]: betweenness_removed_ba_graphs = generate_target_betweenness centrality_removed_graphs(
graphs=ba_graphs_samples,
percentages_to_be_removed=PERCENTAGES_TO_BE_REMOVED,
top_n_to_be_removed=TOP_N_TO_BE_REMOVED)
plot_ba_data_betweenness = generate_plot_data(betweenness_removed_ba_graphs)
plot_removed_data(plot_ba_data_betweenness, 'BA-model: Betweenness removal of
different nodes')
```



```
[369]: #plot_data_random
```

```
[370]: #plot_data_degree
```

```
[371]: #plot_data_betweenness
```

```
[372]: eu_airlines_plot_data = {
    'random': plot_data_random['graph_eu_airlines'],
    'degree': plot_data_degree['graph_eu_airlines'],
    'betweenness': plot_data_betweenness['graph_eu_airlines']
}
```

```
[373]: #eu_airlines_plot_data
```

```
[374]: power_plot_data = {
        'random': plot_data_random['graph_power'],
        'degree': plot_data_degree['graph_power'],
        'betweenness': plot_data_betweenness['graph_power']
    }
```

```
[375]: #power_plot_data
```

```
[376]: #plot_er_data_random
```

```
[377]: er_plot_data = {
        '1k-1': {
            'random': plot_er_data_random['1k-1'],
            'degree': plot_er_data_degree['1k-1'],
            'betweenness': plot_er_data_betweenness['1k-1']
        },
        '4k-1': {
            'random': plot_er_data_random['4k-1'],
            'degree': plot_er_data_degree['4k-1'],
            'betweenness': plot_er_data_betweenness['4k-1']
        }
    }
```

```
[378]: #er_plot_data
```

```
[379]: #plot_ba_data_random
```

```
[380]: ba_plot_data = {
        '2m-1': {
            'random': plot_ba_data_random['2m-1'],
            'degree': plot_ba_data_degree['2m-1'],
            'betweenness': plot_ba_data_betweenness['2m-1']
        },
        '4m-1': {
            'random': plot_ba_data_random['4m-1'],
            'degree': plot_ba_data_degree['4m-1'],
            'betweenness': plot_ba_data_betweenness['4m-1']
        }
    }
```

```
[381]: def plot_attacks(data_dict):
        plt.figure(figsize=(10, 6))

        for key, value in data_dict.items():
            for inner_key, inner_value in value.items():
                x_data = inner_value['percentage']['x_percentages_removed']
                y_data = inner_value['percentage']['y_size_lcc_frac']
```

```

plt.plot(x_data, y_data, label=f'{key}_{inner_key}')

plt.xlabel('x_percentages_removed')
plt.ylabel('y_size_lcc_frac')
plt.title('Percentage Removed vs Fraction of Largest Connected Component')
plt.legend()
plt.grid(True)
plt.show()

```

```

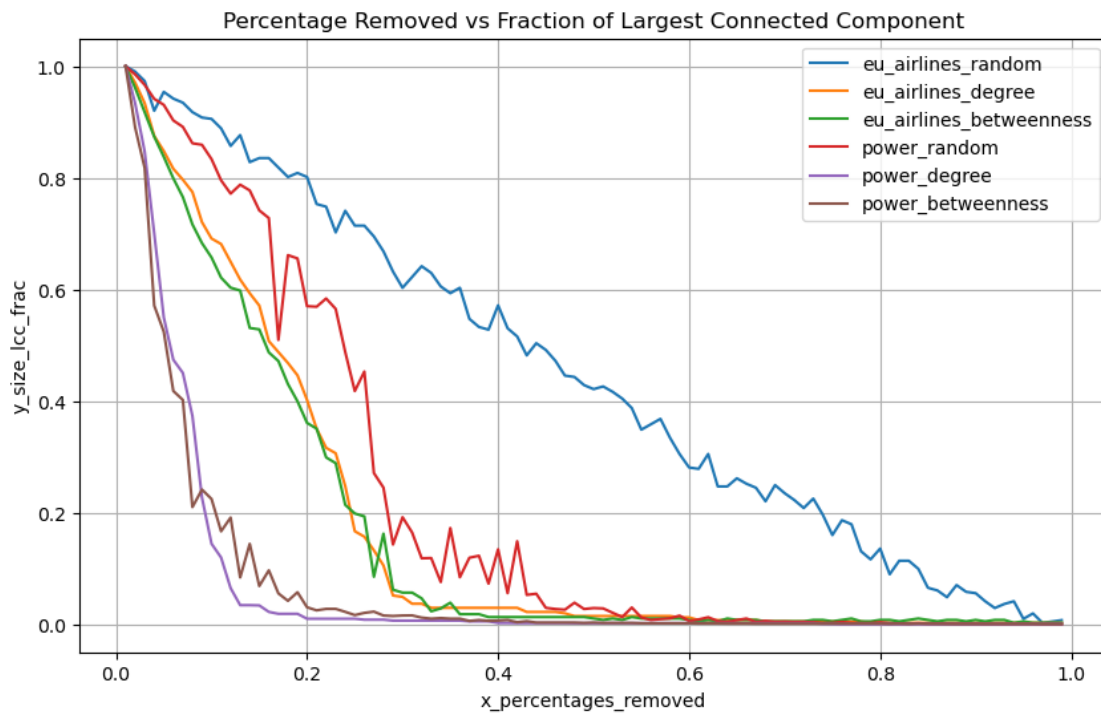
[382]: plot_data_real_networks = {
        'eu_airlines': eu_airlines_plot_data,
        'power': power_plot_data
    }

```

```

[383]: plot_attacks(plot_data_real_networks)

```



```

[384]: def plot_attacks(plot_data, plot_name):
        plt.figure(figsize=(10, 6))
        for inner_key, inner_value in plot_data.items():
            x_data = inner_value['percentage']['x_percentages_removed']
            y_data = inner_value['percentage']['y_size_lcc_frac']
            plt.plot(x_data, y_data, label=f'{inner_key} attack', alpha=0.8)

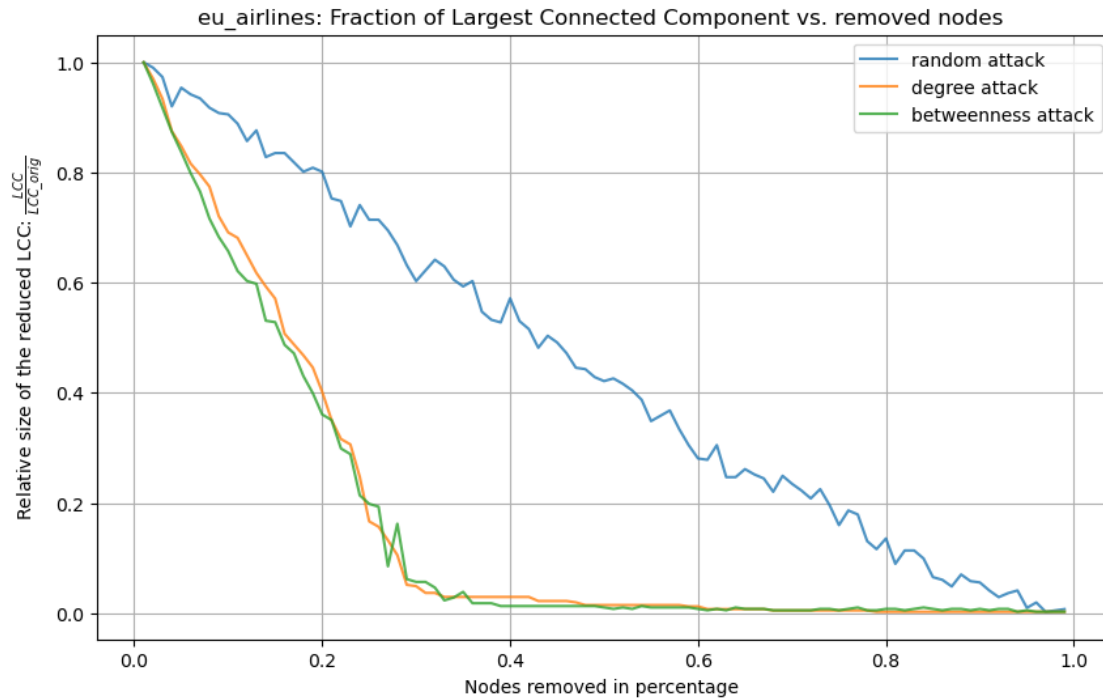
```

```

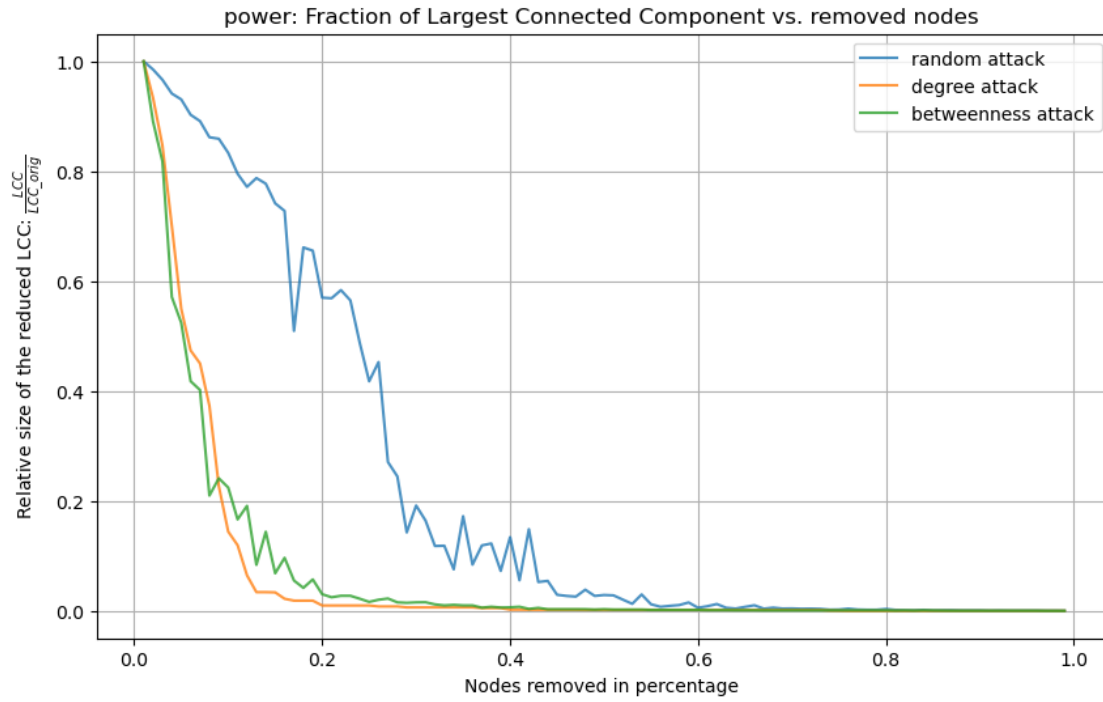
plt.xlabel('Nodes removed in percentage ')
plt.ylabel(fr'Relative size of the reduced LCC:  $\frac{LCC}{LCC_{orig}}$ ')
plt.title(f'{plot_name}: Fraction of Largest Connected Component vs. removed nodes ')
plt.legend()
plt.grid(True)
plot_name_formatted=plot_name.replace('$', '')
plot_name_formatted=plot_name_formatted.replace(' ', '_')
plot_name_formatted=plot_name_formatted.lower()
file_name=f'{plot_name_formatted}-attacks.png'
complete_plot_filename=os.path.join(OUTPUT_PATH_NODE_REMOVALS, file_name)
plt.savefig(complete_plot_filename)
plt.show()

```

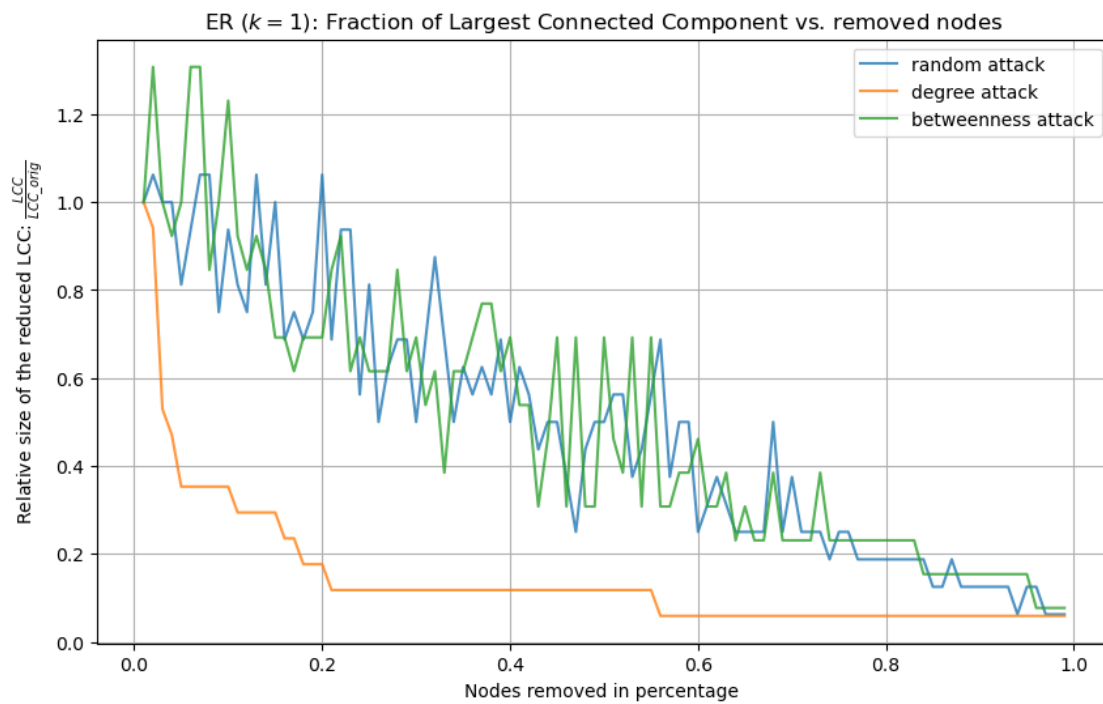
```
[385]: plot_attacks(plot_data_real_networks['eu_airlines'], 'eu_airlines')
```



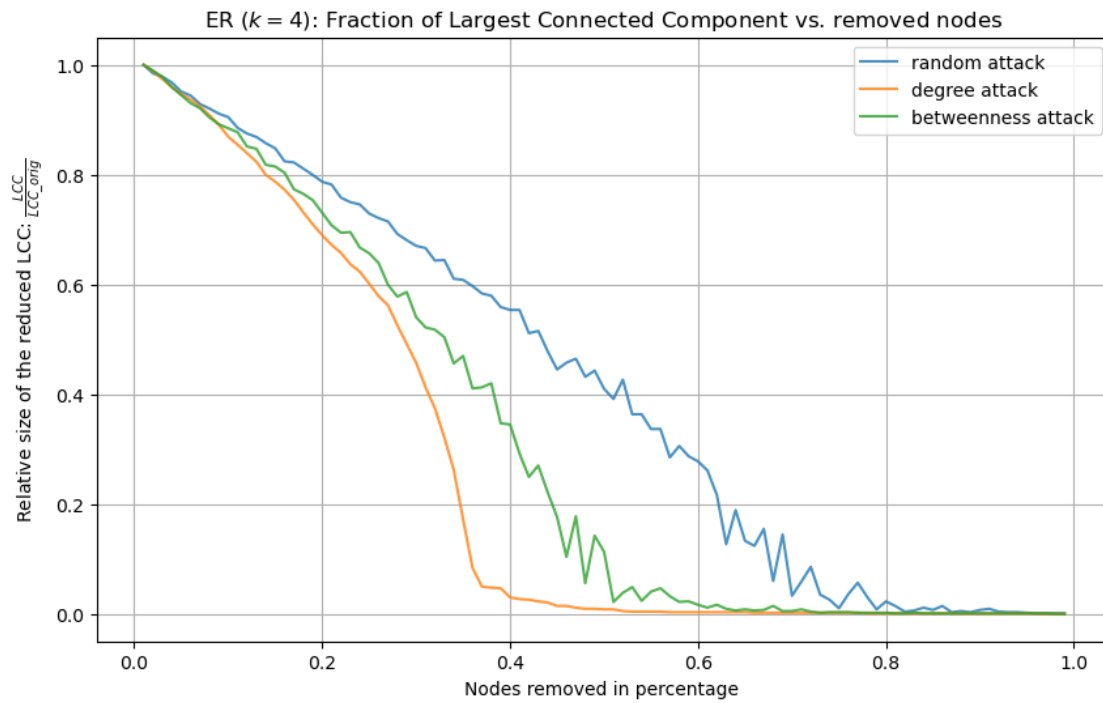
```
[386]: plot_attacks(plot_data_real_networks['power'], 'power')
```

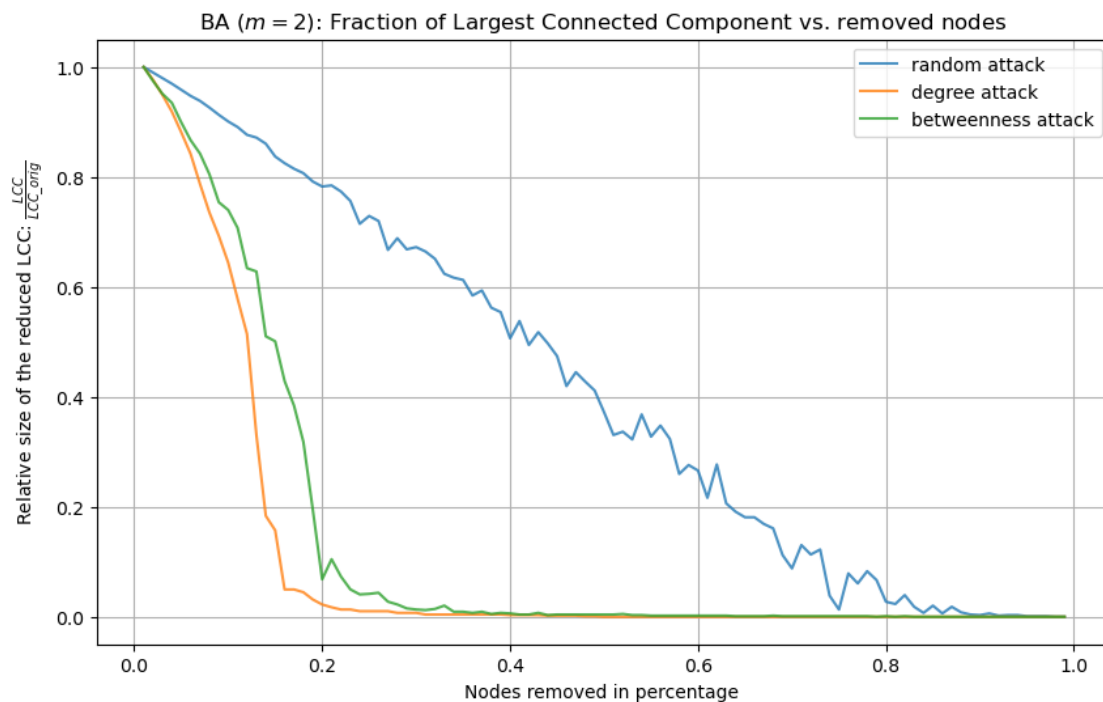
```
[387]: plot_attacks(er_plot_data['1k-1'], fr'ER ($k=1$)')
```



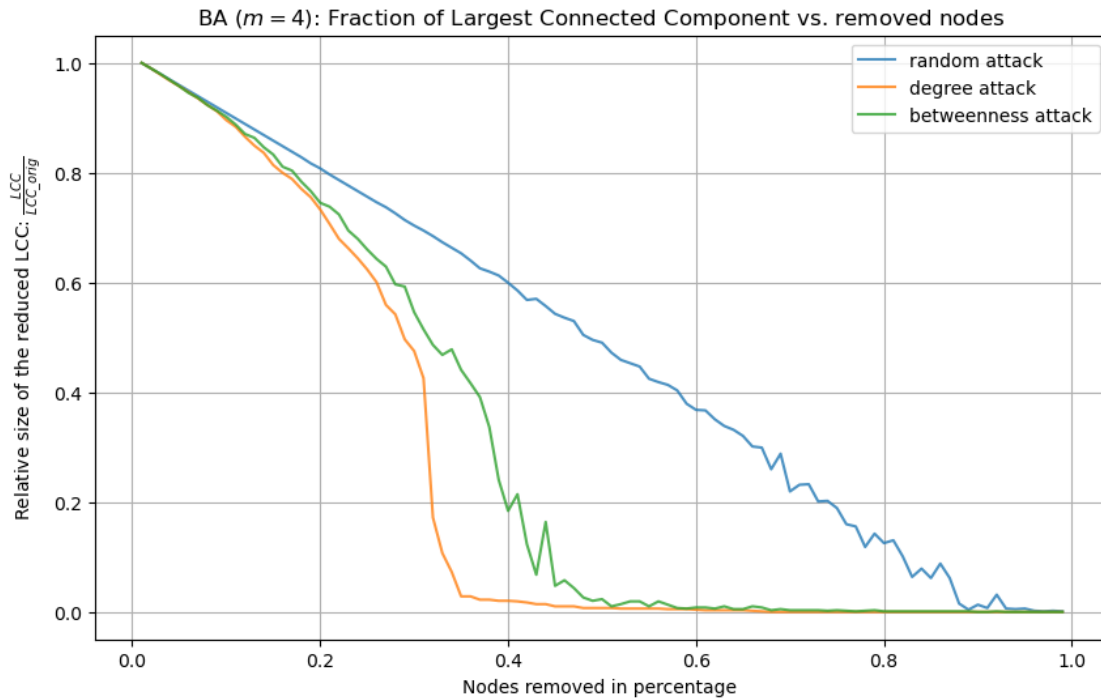
```
[388]: plot_attacks(er_plot_data['4k-1'], fr'ER ($k=4$)')
```



```
[389]: plot_attacks(ba_plot_data['2m-1'], fr'BA ($m=2$)')
```



```
[390]: plot_attacks(ba_plot_data['4m-1'], fr'BA ($m=4$)')
```



2.4 4. Comment the plots, using what we discussed in class.

Generally, we're dealing with real-world scale-free networks (airlines, power), two instances of random networks (ER with different k values) and two instances of synthetic scale-free networks (BA with different m values). We can confirm what fact from lecture that "Scale-free networks are more fragile to targeted attacks and their functioning deteriorates more rapidly than random networks". Evidence for this are: - *eu_airlines*: the degree and betweenness attack do show a more drastic decrease of the relative LCC compared to the random attack. (at. $x=0.2$, $y=0.4$ for targeted, respectively $y=0.8$ when random) - *power*: the degree and betweenness attack do show a more drastic decrease of the relative LCC compared to the random attack. (at. $x=0.2$, y is almost zero for targeted, respectively $y=0.6$ when random) - *BA-model*: we observe a similar behaviour to the aforementioned. We also can see that the decrease is more when m is smaller which is to be expected. - *ER-model*: for a random model with $k=1$, we observe a very unstable behaviour in decreasing nodes for random and betweenness attack, whereas the degree attack does constantly decrease the relative LCC value. The larger k (e.g., 4) the more similar will the behaviour of the different attacks be. This is also to be expected.

Creating Results: 'jupyter nbconvert Assignment_03.ipynb -to pdf -output Assignment_03_Results-T3.pdf

```
[391]: import os
os.system('jupyter nbconvert Assignment_03.ipynb --to pdf --output_
↪Assignment_03_Results-T3.pdf')
```

```
[NbConvertApp] Converting notebook Assignment_03.ipynb to pdf
[NbConvertApp] Support files will be in Assignment_03_Results-T3_files/
[NbConvertApp] Making directory ./Assignment_03_Results-T3_files
[NbConvertApp] Writing 122706 bytes to notebook.tex
[NbConvertApp] Building PDF
[NbConvertApp] Running xelatex 3 times: ['xelatex', 'notebook.tex', '-quiet']
[NbConvertApp] Running bibtex 1 time: ['bibtex', 'notebook']
[NbConvertApp] WARNING | bibtex had problems, most likely because there were no
citations
[NbConvertApp] PDF successfully created
[NbConvertApp] Writing 175299 bytes to Assignment_03_Results-T3.pdf
```

```
[391]: 0
```